SigNet's Public-to-Private Cross-Chain Transaction System

Formal Treatment of its Security Analysis

NOVEMBER 10, 2026

Alessandra Scafuro, alessandra@sig.network

Abstract

This document establishes the foundation for analyzing the soundness and unlinkability of SigNet's Public-to-Private Cross-Chain System. The system comprises several complex components: the MPC systems, the Midnight shielded-transaction layer and shielded-computation layer, the Ethereum ledger. After establishing the foundations, this document focuses exclusively on the vault contract at release tag `erc20-vault-v0.3.0`[1]. (For a high-level description of the system logic consult the ERC20 Vault readme of that release.)

Since the vault is written in compact language (provided by Midnight) and it relies on primitives provided by the Midnight shielded-transaction layer, our proof explicitly identifies the cryptographic properties that we assume from the Midnight components, though we do not formally prove that the Mindnight components achieve them. The document is organized as follows.

1. Section 2: Formal definition of Public-to-Private Cross-chain Payment System. This section establishes a formal model for a Public-to-Private Cross-Chain Transaction System over two abstract ledgers: a public ledger ($\mathcal{L}_{\mathsf{pub}}$) and a private ledger ($\mathcal{L}_{\mathsf{shd}}$). It defines core operations (e.g., Deposit, Withdraw, Swap, Supply, Redeem) and their inputs/outputs description, providing a generalized framework adaptable to any pair of chains.
2. Section 3: Formal definition of *Unlinkability* in a *Public-to-Private* Cross-chain Transaction System. This section expresses the unlinkability that a public-to-private chain system can achieve via a Game-Based definition, standard in cryptographic proofs. We preferred the game-based approach over the simulation-based approach in the Universal Composability Framework for simplicity. We believe that the game clearly identifies what is protected (and what is leaked) in a public-to-private chain system.
3. Section 4 and Section 5: Implementation. We provide an abridged description of the vault contract (release `erc20-vault-v0.3.0`). Our implementation uses shielded-transaction layer and shielded-computation layer provided by Midnight. When useful, we use blue-shaded boxes to highlight that we are using a midnight component, and therefore we rely on its security and soundness. Furthermore, our contract uses the SygNet's MPC signing system that is treated here as a *trusted signing system*. We also assume that all components related to emitting and processing chain events are trusted.
4. Section 6: Assumptions. We state the cryptographic assumptions we use and the ones that we inherit from the Midnight private circuit-layer and shielded-coin layer.
5. Section 7: A semi-formal analysis of the security of implementation. We prove that our cross-chain system achieves the unlikability properties (under the assumption that the network is anonymized) and the soundness property. In each section, we state the model and cryptographic assumptions we require for the proof.

[1] This is an **earlier version** of the system. The latest release, `erc20-vault-v0.4.0`, contains modifications that are not formally analysed here. Section 1 lists them. Nevertheless, the proof strategy used in this paper should extend to the latest release.

6. Section 9: Conclusions. Finally, we distill essential takeaways from this treatment. We explain what our analysis means for a user.

1 VERSION ANALYSED IN THIS DOCUMENT

This document analyses the vault contract at release tag `erc20-vault-v0.3.0`. The contract source at that tag is byte-identical to the source at the earlier tags `erc20-vault-v0.1.0` and `erc20-vault-v0.2.0`, so the analysis covers those releases as well. Every circuit, ledger field and source identifier quoted in this document is taken from that version, which prevents replay attacks and keeps request ids distinct with a global request counter (reqCounter).

The latest release is `erc20-vault-v0.4.0`. It is not formally analysed here. It drops the global counter in favour of a design based on timing information (a block-height watermark, `globalLastSeen`, raised by every attestation the vault folds in) and introduces a request queue that lets many requests be processed in parallel while preventing contention: a request is started by its caller, assigned a nonce and sent to the MPC by permissionless circuits, and settled by its caller once a permissionless circuit has queued the MPC's attestation. The refund circuits are folded into the completion circuits, the per-request authentication marker becomes a keyed commitment over a caller-chosen random index in place of the request id, and deployer-gated administration circuits (gas settings, an allow-list of deposit and swap tokens, the approvals and an emergency nonce replacement) are added.

The public procedures (circuits) the latest release introduces to organise requests sent to the MPC and attestations received from it do not use any private inputs, can be executed by anyone and are anonymous (i.e., the identity of the caller is not leaked by the circuit computation). As such, they do not affect the rationale behind the unlinkability proofs provided in this document. The soundness approach is expected to remain unchanged, but this must be confirmed with a formal security proof.

In the document, we will use the paragraph-header: " **Differences with the latest release (erc20-vault-v0.4.0):** ..." to describe the differences with the latest release.

2 PRIVATE-PUBLIC CROSS-CHAIN TRANSACTION SYSTEM: FORMAL DEFINITION

This section describes the functionality that a Private-Public Cross-chain Transaction System provides formally. The system is defined in terms of the commands it provide. Each command is defined via the input-output interface.

A cross-chain transaction system is a bridge between a public EVM-ledger $\mathcal{L}_{\text{pub}}$ (e.g., Ethereum) and a private (shielded) ledger $\mathcal{L}_{\text{shd}}$ (e.g., Midnight). The users of the cross-chain system are users of possess account on both chains. The system and the users interact with each chain using the available interface.

Definition 1 (Public-to-Private Cross-chain Transaction System). A *public-to-private cross-chain defi system* Π, relative to $\mathcal{L}_{\text{pub}}$ and $\mathcal{L}_{\text{shd}}$, consists of the following PPT procedures. Every procedure has oracle access to $\mathcal{L}_{\text{pub}}$ and $\mathcal{L}_{\text{shd}}$.

Each procedure outputs the following tuple:

$$\big(\,\text{state}'_{\text{pub}},\ \text{state}'_{\text{shd}},\ \text{transcript}, \text{outcome};\ \text{privOut}\,\big),$$

where $\text{state}'_{\text{pub}}$, $\text{state}'_{\text{shd}}$ are the resulting state updates for each ledger; transcript is the public transcript the execution of the call produces; outcome indicates if the function terminates successfully, or it failed (functions are not guaranteed to terminate) and privOut is a private output the caller retains.

1. $(\text{pp}, \rho) \leftarrow \Pi.\text{Setup}^{\mathcal{L}_{\text{pub}},\mathcal{L}_{\text{shd}}}(1^{\lambda})$
   Initialises the system on both ledgers and outputs public parameters pp. Run once, by a trusted party. pp **is an implicit input to every procedure below,** and fixes the rules and parameters of the cross-chain payment system.
2. $\Pi.\text{Deposit}^{\mathcal{L}_{\text{pub}},\mathcal{L}_{\text{shd}}}\big([\text{pubAddr}],\ \text{shdAcct},\ \text{tokType},\ \text{amt},\ \text{aux}\,;\ \text{privIn}\big)$.
   Called by the holder of the shdAcct, whose secret is in privIn, aux carries execution parameters of the chains. On succeeded: amt units of tokType is moved to a vault on $\mathcal{L}_{\text{pub}}$ and credited to shdAcct. On failed no operation is performed. pubAddr is the address on $\mathcal{L}_{\text{pub}}$ that funded the deposit to the vault. The parenthesis [] emphasize that shdAcct doesn't need to be the holder of pubAddr – though in the practical instantiation it is [2].
3. $\Pi.\text{Withdraw}^{\mathcal{L}_{\text{pub}},\mathcal{L}_{\text{shd}}}\big(\text{shdAcct},\ \text{destAddr}*,\ \text{tokType},\ \text{amt},\ \text{aux}\,;\ \text{privIn}\big)$.
   Called by the holder of the shielded-chain account shdAcct, whose secret is in privIn. If shdAcct owns at least amt units of tokType on the vault, amt units of tokType leave shdAcct on $\mathcal{L}_{\text{shd}}$ and amt units are credited to destAddr on $\mathcal{L}_{\text{pub}}$. On failed: shdAcct's balance is unchanged.
4. $\Pi.\text{Swap}^{\mathcal{L}_{\text{pub}},\mathcal{L}_{\text{shd}}}\big(\text{shdAcct},\ \text{tokType},\ \text{amt},\ \text{tokType}',\ \text{amt}',\ \text{aux}\,;\ \text{privIn}\big)$.
   Called by the holder of shdAcct. On succeeded: amt units of tokType are swapped with amt' units of $\text{tokType}'$. On failed: shdAcct's balance is unchanged.
5. $\Pi.\text{Supply}^{\mathcal{L}_{\text{pub}},\mathcal{L}_{\text{shd}}}\big(\text{shdAcct},\ [\text{tokType}, \text{tokType}'],\ \text{amt},\ \text{aux}\,;\ \text{privIn}\big)$.
   Called by the holder of the shielded-chain account shdAcct, whose secret is in privIn. If shdAcct owns at least amt units of tokType on the vault, then on succeeded: amt units of tokType leave shdAcct on $\mathcal{L}_{\text{shd}}$, the vault supplies them to the lending market on $\mathcal{L}_{\text{pub}}$, and amtShares units

[2] In our implementation, an address pubAddr must send funds to an intermediate, application-derived account, that is controlled by the Bridge. Such intermediate account is the used to deposit the amount amt to the vault address. We explicitly list pubAddr as a public genesis address (independent on the implementation), to stress that the link between the accounts on the two ledgers is revealed by a deposit

of tokType′ are credited to shdAcct. On failed: shdAcct's balance is unchanged. amtShares is determined by the state of $\mathcal{L}_{\text{pub}}$ at the moment of execution, and is reported back in transcript. [tokType, tokType′] are optional parameters (our implementation of Supply they are fixed and defined in the initial parameters pp.).

6. $\Pi.\text{Redeem}^{\mathcal{L}_{\text{pub}},\mathcal{L}_{\text{shd}}}(\text{shdAcct}, [\text{tokType}'], \text{amtShares}, \text{aux}\,;\,\text{privIn})$.
Called by the holder of shdAcct. If shdAcct owns at least amtShares units of tokType′ on the vault, then on succeeded: amtShares units of tokType′ leave shdAcct on $\mathcal{L}_{\text{shd}}$, the vault redeems them on $\mathcal{L}_{\text{pub}}$, and amtAssets units of tokType (principal plus accrued interest) are credited to shdAcct. On failed: shdAcct's balance is unchanged.

3 DEFINITION OF UNLINKABILITY.

A Public-to-Private Cross-chain Transaction System is designed for actions to be initiated on a private chain and to have effect on a vault on the public chain. The latter, requires that amounts of the transactions be revealed. With that in mind, there are two notions of unlinkability that one could aim for:

- *Transaction unlinkability*: can an observer tie a withdrawal to a user? This connection could depend on amounts and on the order of events, and cannot be a property of a protocol: if the public chain records that the vault received only two deposits, one of 10 from pubAddr_A and one deposit of 6 from pubAddr_B later a withdrawal of 7 to a fresh new account pubAddr*, leaks that the withdrawal request came from the user controlling pubAddr_A, regardless of the cryptography involved in the withdrawal process.
- *Cryptographic unlinkability*: do the protocol messages give any *additional* help, besides what can be inferred from the public amounts displayed on the public chain?

In this section we target the latter. We provide the notion of *Withdrawer's Unlinkability* that states that it should be computationally infeasible from any PPT adversary to guess the identity of the shielded account associated to a withdrawal, even given full knowledge of the deposit and withdrawal/swaps/supply/redeems performed by all honest users (and even actively participating as a malicious user in the system). We capture this definition with a formal cryptographic game, the ithdrawer unlinkability game $\text{Exp-Unlink}_{\Pi,\mathcal{A}}(1^\lambda)$.

In the experiment, there is a challenger that (honestly) simulates the cross-chain system and the two ledgers for the adversary. The adversary can choose the operations that are executed in the cross-chain system: it can create honest accounts, and make them perform any action (i.e., deposits, withdrawals, swaps, supply redeem) on amount and types of her choice. In the formal game this is modeled by the fact that the adversary has oracle access to each action provided by the system and by the two chains. The oracles are described in Section 3.1.

At the challenge phase, the adversary selects two shielded accounts and one amount v. If both accounts have at least a balance of v, the challenger will choose one at random and run the withdrawer function function on it, revealing the resulting transcript to the adversary. The goal of the adversary now is to guess which bit was chosen by the challenger. Before outputting his guess, the adversary can continue interact with the system arbitrarily, and the system will respond to the request *under the condition that both accounts in the challenge phase have a $-v$ amount in their balance.* The adversary wins the game if it guesses correctly with a probability better than half.

The power of the adversary. We consider an adversary that has full view of all public protocol messages and Ledger's messages. Furthermore, we consider an adversary that can have any type of side channel information about the users. For instance, the adversary could know that a certain action (e.g., Withdraw, Deposit, Supply, etc) was issued by a specific user. Formally, we model this knowledge by giving to the adversary oracle access to all the calls Deposit, Withdraw, Swap, Supply, Redeem provided by our interface. The adversary can decide which users perform which action and choose the amount,. Furthermore, the adversary is always aware of all operations that take place to each ledger. For the shielded ledger, the adversary could have side channel information about the user who is performing a certain shielded transaction at a certain point in time. Once again, we model this knowledge by giving the adversary the power to choose which user performs which operations on each chain. This power is modeled by the Oracles (§ 3.1).

Remark 1 (No Network Information)**.** We assume an adversary that collects information solely at the application layer and explicitly exclude network-layer threats (e.g., user IP addresses, routing metadata). Since network-level data can deanonymize messages regardless of application-layer cryptographic protections, restricting the adversary's scope to the application layer is a standard assumption in privacy-preserving systems.

3.1 Oracles

As mentioned above, in our security model we consider an adversary who could have side channel information about certain users taking certain actions at certain time, even in the shielded ledger. To formally modeled this in our security experiment, we give the adversary access to oracles to call actions for the cross-chain system, and oracles to the ledgers. In the security experiment, the *challenger* simulates the oracle cross-chain system and the ledgers for the adversary. The challenger $\mathcal{C}$ maintains the relevant states to simulate the various systems, and in particular, it maintains a balance sheet $\mathsf{vaultBal}[\cdot][\cdot]$, recording for every account it created how much of each type that account holds, updated on every call; both ledgers; and the transcript $T = (T_{\mathsf{pub}}, T_{\mathsf{shd}})$. Every oracle call appends to T everything the two ledgers publish because of it, and $\mathcal{A}$ reads T at all times.

3.1.1 Ledger oracles.

The adversary $\mathcal{A}$ has the power to create new ledger users (for both public and private ledger), and to instruct any operation allowed on each chain and observe the result of those activities. This power is captured by giving the adversary oracle calls to actions (e.g., Create, Mint, Transfer, etc.) over $\mathcal{L}_{\mathsf{pub}}$ and $\mathcal{L}_{\mathsf{shd}}$. Below we describe the ledger oracles provided to the adversary.

Oracles for the public ledger $\mathcal{L}_{\mathsf{pub}}$,

- $\mathcal{O}\mathsf{NewPubAcct}() \rightarrow \mathsf{pubAddr}_i$: it allows to create new account on the public chain. The ledger creates it, $\mathcal{C}$ retains the account secret, and the public address created $\mathsf{pubAddr}_i$ is returned to $\mathcal{A}$.
- $\mathcal{O}\mathsf{TransferPub}(\mathsf{pubAddr}, \mathsf{pubAddr}', \mathsf{tokType}, \mathsf{amt})$ it allows to request a public-chain transfer from pubAddr to pubAddr′ of amt of tokType.
- $\mathcal{L}_{\mathsf{pub}}.\mathsf{Execute}(\mathsf{pubAddr}, \mathsf{contract}, \mathsf{input})$: moves amt units of tokType
- $T_{\mathsf{pub}} \leftarrow \mathcal{L}_{\mathsf{pub}}.\mathsf{Read}()$ It returns the full history of the ledger.

Oracles for the shielded ledger $\mathcal{L}_{\mathsf{shd}}$,

- $\mathcal{O}$NewShdAcct$() \rightarrow$ shdAcct$_i$, it allows to create new account on the shielded chain. The ledger creates it, $\mathcal{C}$ retains the account secret, and the public part of the shielded address shdAcct$_i$ is returned to $\mathcal{A}$.
- $\mathcal{O}$Mint(shdAcct$_i$, tokType, amt): it allows to instruct a transfer amt to the private pool, for account shdAcct$_i$, on $\mathcal{L}_{\text{shd}}$. The secret associated to this call are maintained by the challenger.
- $\mathcal{O}$TransferShd(shdAcct$_i$, shdAcct$_j$, tokType, amt): this oracle allows a *public* transfer on $\mathcal{L}_{\text{shd}}$.
- $\mathcal{O}$ShieldedTransfer(shdAcct, shdAcct$'$, tokType, amt): this oracle allows $\mathcal{A}$ to request a *shielded* transfer on $\mathcal{L}_{\text{shd}}$ on accounts of their choice shdAcct, shdAcct$'$.
- $\mathcal{L}_{\text{shd}}$.Execute(shdAcct$_i$, contract, input): this oracle call allows $\mathcal{A}$ to request execution of contract on behalf of shdAcct$_i$ on public input input. The private input is maintained by the challenger.
- $T_{\text{shd}} \leftarrow \mathcal{L}_{\text{shd}}$.Read() It returns the full history of the ledger.

Separately, $\mathcal{A}$ may create and operate accounts of its own directly through the ledger interfaces, on either chain, and post its own transactions. The oracles above exist so that $\mathcal{A}$ can also direct accounts whose secrets it does *not* hold. We call an account *honest* if it was created by the oracles.

3.1.2 Cross-chain oracles.

Using the cross-chain oracles the adversary can orchestrate the interactions between honest parties, choosing which parties need to perform which operations, choosing amounts and auxiliary information aux (such as gas prices etc). The cross-chain oracles are described below. The adversary decides the inputs on which the oracles to be run on. The challenger then honestly executes those calls on behalf of honest users (i.e., the challenge knows the private state of honest users) and the resulting transcripts are returned to the adversary.

- $\mathcal{O}$Deposit(pubAddr, shdAcct, tokType, amt, aux, outcome): The challenger runs Deposit on behalf of shdAcct, and using pubAddr as the source account on the public chain. On succeeded, add amt to vaultBal[shdAcct][tokType] and update the ledger balance of pubAddr.
- $\mathcal{O}$Withdraw(shdAcct, destAddr$*$, tokType, amt): runs Withdraw on behalf of the holder of shdAcct, provided vaultBal[shdAcct][tokType] $\geq$ amt. On succeeded, moves amt from vaultBal[shdAcct][tokType] to ledgerBal[pubAddr][tokType].
- $\mathcal{O}$Swap(shdAcct, tokType, amt, recvAcct, tokType$'$, amt$'$, aux): runs Swap on behalf of the holder of shdAcct, provided vaultBal[shdAcct][tokType] $\geq$ amt. On succeeded, subtracts amt from vaultBal[shdAcct][tokType] and adds amt$'$ to vaultBal[recvAcct][tokType$'$].
- $\mathcal{O}$Supply(shdAcct$_i$, tokType, amt, aux, outcome): runs Supply on behalf of the holder of shdAcct, provided vaultBal[shdAcct][tokType] $\geq$ amt. On succeeded, subtracts amt from vaultBal[shdAcct][tokType] and adds amtShares to vaultBal[shdAcct][tokType$'$], where amtShares is the amount $\mathcal{L}_{\text{pub}}$ returned and is recorded in T, and tokType$'$ is the exchange token defined by the Supply protocol. On failed, no balance changes.
- $\mathcal{O}$Redeem(shdAcct$_i$, , amtShares, aux, outcome): runs Redeem on behalf of the holder of shdAcct, provided vaultBal[shdAcct][tokType$'$] $\geq$ amtShares. On succeeded, subtracts amtShares from vaultBal[shdAcct][tokType$'$] and adds amtAssets to vaultBal[shdAcct][tokType], with amtAssets as returned by $\mathcal{L}_{\text{pub}}$ and recorded in T. On failed, no balance changes.
  Note. The amtAssets is a parameter dictated by $\mathcal{L}_{\text{pub}}$ (and not an input to the oracle). This reflect the scenario where the exchange rate is defined by the market, and it is independent on by the user who attempts to redeem.

◇ Future definitions could model a scenario where additional criteria are used (e.g., a user must prove that it belong to a whitelist)

A call whose stated balance condition fails is not executed and $\mathcal{C}$ returns $\perp$.

3.2 Withdrawer unlinkability Definition

3.2.1 Experiment Exp-Unlink

Exp-Unlink$_{\Pi,\mathcal{A}}(1^\lambda)$

$\mathcal{A}$ has access to the ledger oracles and cross-chain oracle defined in §3.1 and simulated by the challenger. Let $\mathcal{A}^{\mathcal{O}(\cdot)}$ denote the adversary having access to such oracles.

- **Initialization.** $(\text{pp}, \rho) \leftarrow \text{Setup}^{\mathcal{L}_{\text{pub}},\mathcal{L}_{\text{shd}}}(1^\lambda)$; vaultBal $:= 0$; $T := \emptyset$. $\mathcal{A}$ is given pp.
- **Pre-challenge Training.** $\mathcal{A}$ has access to the oracles described above:

$$(\text{shAcct}_0,\ \text{shAcct}_1,\ \text{tokType},\ \text{amt},\ \text{aux}^*,\ \text{outcome}^*,\ \text{st}) \leftarrow \mathcal{A}^{\mathcal{O}}(\text{pp}),$$

  where $\text{shAcct}_0, \text{shAcct}_1$ are honest shielded accounts (created through the $\mathcal{O}$NewShdAcct oracle.
- **Challenge**
  - if shAcct_0 or shAcct_1 is not honest, output 0; // C1. if one user is controlled by the adversary, distinguishing is trivial
  - if $\text{vaultBal}[\text{shAcct}_0][\text{tokType}] < \text{amt}$ or $\text{vaultBal}[\text{shAcct}_1][\text{tokType}] < \text{amt}$, output 0; // If the amounts are different, the adversary can easily tell from amount posted on the public chain $\mathcal{L}_{\text{pub}}$.
  - otherwise:
    1. sample $b \leftarrow \{0,1\}$;
    2. create a fresh public-chain address destAddr* and execute

$$\text{Withdraw}(\text{shAcct}_b, \text{destAddr}^*, \text{tokType}, \text{amt}, \text{aux}\,;\, \text{privIn}_b)$$

       with the output

$$\text{state}_{\text{pub}}{*}',\ \text{state}_{\text{shd}}{*}',\ \text{transcript}^*, \text{outcome}^*$$

       returned to $\mathcal{A}$.
    3. if outcome$^* =$ succeeded:
       Update the balance state for both account:
       $\text{vaultBal}[\text{shAcct}_0][\text{tokType}] -= \text{amt}$ *and* $\text{vaultBal}[\text{shAcct}_1][\text{tokType}] -= \text{amt}$;
       and use these amounts to simulate all the oracles § 3.1.2,§ 3.1.1.
    4. if outcome $=$ failed balances are not updated.
- **Post-Challenge Training.** $\mathcal{A}$ continues to have access to all the oracles above, with $\mathcal{C}$ answering every cross-chain oracle calls against the balance sheet vaultBal updated in the challenge phase.

$$b' \leftarrow \mathcal{A}^{\mathcal{O}}(\text{st}, T)$$

- **Output** The output of the experiment is 1 iff $b = b'$.

Fig. 1

Definition 2 (Withdrawer unlinkability)**.** A cross-chain payment protocol has Π satisfies *withdrawer*

unlinkability if for all PPT $\mathcal{A}$ there is a negligible μ such that

$$\Pr[\text{Exp-Unlink}_{\Pi,\mathcal{A}}(1^\lambda) = 1] \leq \frac{1}{2} + \mu(\lambda)$$

Remark 2 (Pending Requests.)**.** In the Unlinkability Experiment we assume that the cross-chain oracle calls that involve *honest parties* always terminate (either with success or failure).

3.3 Swapper, Supplier, Swapper and Redeemer Unlinkability

The exact same experiment can be applied to every action of the cross-chain transaction system. Write act $\in \{\text{Swap}, \text{Supply}, \text{Redeem}\}$, and for a given action act let $\text{tokType}_{\text{in}}$ and $\text{tokType}_{\text{out}}$ be the input and output types. Let amt be the input amount and let amt^{out} denote the amount $\mathcal{L}_{\text{pub}}$ returns, which the challenger learns when it runs the call. We provide general the experiment in Figure 2.

$\text{Exp-WithdrawUnlink}_{\Pi,\mathcal{A}}(1^\lambda)$

$\mathcal{A}$ has access to all oracles of §3.1.

- **Initialization.** $(\text{pp}, \rho) \leftarrow \text{Setup}^{\mathcal{L}_{\text{pub}}, \mathcal{L}_{\text{shd}}}(1^\lambda)$; $\text{vaultBal} := 0$; $T := \emptyset$. $\mathcal{A}$ is given pp.
- **Pre-challenge Training.** $\mathcal{A}$ has access to the cross-chain oracles described above (as well as the ledger oracles).

  $$(\text{act}, \text{shAcct}_0, \text{shAcct}_1, \text{tokType}_{\text{in}}, \text{amt}, \text{tokType}_{\text{out}}, \text{aux}^*, \text{outcome}^*, \text{st}) \leftarrow \mathcal{A}^{\mathcal{O}}(\text{pp}),$$

  where act $\in \{\text{Withdraw}, \text{Swap}, \text{Supply}, \text{Redeem}\}$, the parameters aux are those the chosen action takes, and $\text{shAcct}_0, \text{shAcct}_1$ are honest shielded accounts.
- **Challenge**
  - if shAcct_0 or shAcct_1 is not honest, output 0;
  - if $\text{vaultBal}[\text{shAcct}_0][\text{tokType}_{\text{in}}] < \text{amt}$ or $\text{vaultBal}[\text{shAcct}_1][\text{tokType}_{\text{in}}] < \text{amt}$, output 0;
  - otherwise:
    1. sample $b \leftarrow \{0, 1\}$;
    2. if act $=$ Withdraw, create a fresh public-chain address destAddr^* and include it in aux;
    3. execute

       $$\text{act}(\text{shAcct}_b, \text{aux}, \text{aux}^*; \text{privIn}_b)$$

       with the output

       $$\text{state}_{\text{pub}}{}^{*\prime}, \text{state}_{\text{shd}}{}^{*\prime}, \text{transcript}^*, \text{outcome}^*$$

       returned to $\mathcal{A}$;
    4. Let $\text{st}^{(0)}$ and $\text{st}^{(1)}$ denote the states of the honest parties in the two worlds: $\text{st}^{(\beta)}$ is the state that results from running act on behalf of shAcct_β. The challenger executes the challenge in world b and **retains both** $\text{st}^{(0)}$ and $\text{st}^{(1)}$.
- **Post-Challenge Training.** $\mathcal{A}$ continues to have access to all the oracles above, with $\mathcal{C}$ answering every cross-chain oracle call as follow:
  - if q is executable in *both* $\text{st}^{(0)}$ and $\text{st}^{(1)}$, execute it in world b and return the resulting transcript;
  - otherwise return $\perp$.

  $$b' \leftarrow \mathcal{A}^{\mathcal{O}}(\text{st}, T)$$
- **Output.** The output of the experiment is 1 iff $b = b'$.

Fig. 2

Definition 3 (Action Unlinkability). Π has *action unlinkability* if for every act $\in \{\text{Swap}, \text{Supply}, \text{Redeem}\}$ and every PPT $\mathcal{A}$ there is a negligible μ with

$$\Pr[\text{Exp-LendUnlink}_{\Pi,\mathcal{A}}(1^\lambda) = 1] \leq \frac{1}{2} + \mu(\lambda).$$

4 OUR IMPLEMENTATION: COMPONENTS

The implementation is the ERC20 Vault contract. Consult the ERC20 Vault README for a high-level description of the contract logic and components.

4.1 System Components

Trusted Bridge SigNet**'s MPC.** The instantiation of the cross-chain system relies on SigNet's MPC system acting as a trusted bridge that listen for events (actions such as Deposit, Swap, etc) on Midnight and executed them on Ethereum. Between a request (e.g., Deposit) and its settlement circuit, three steps run outside any proof : the committee signs the requested EVM transaction; a relayer broadcasts it; the committee observes the result and posts a signed attestation of it back to Midnight.

SigNet's MPC Signing System is trusted to satisfy:

1. **MPC committee integrity:** the committee signs only what the protocol authorises.
2. **Attestation faithfulness:** MPC only provide attestations for events that have been executed on $\mathcal{L}_{\text{pub}}$ (Ethereum). The MPC attests failed only if the transaction did not execute.

.

Definition	Instantiated by	Notes
$\mathcal{L}_{\text{pub}}$	Ethereum	accounts are EVM addresses; types are ERC20 contracts; everything public
$\mathcal{L}_{\text{shd}}$	Midnight	enables private payments and execution of contract with private inputs.
SigNet	SigNetwork MPC	It manages the vault account on Ethereum and provides the attestations for it.

4.1.1 Key derivation

SigNet holds a single root key pair $(\text{sk}_{\text{root}}, \text{mpcRootPk})$ on secp256k1, with $\text{mpcRootPk} = \text{sk}_{\text{root}} \cdot G$ for the group generator G of order n. Every account it signs with is an *additively derived* child of that root, scoped by the requesting contract and by a 32-byte path.

Definition 4 (Additive key derivation). Let $\varepsilon : \{0,1\}^* \to \mathbb{Z}_n$ be

$$\varepsilon(\text{vaultAddr}, \text{keyPath}) \;=\; \text{HashP}\big(\,\text{prefix} \,\|\, \text{chainId} \,\|\, \text{vaultAddr} \,\|\, \text{keyPath}\,\big) \bmod n,$$

The derived key pair is

$$\mathsf{sk}_{\mathsf{der}} \;=\; \mathsf{sk}_{\mathsf{root}} + \varepsilon(\mathsf{vaultAddr}, \mathsf{keyPath}) \bmod n,$$
$$\mathsf{Derive}(\mathsf{mpcRootPk},\ \mathsf{vaultAddr},\ \mathsf{keyPath}) \;=\; \mathsf{mpcRootPk} + \varepsilon(\mathsf{vaultAddr}, \mathsf{keyPath}) \cdot G.$$

Where an EVM account is wanted, userDerivedAddr is the standard address encoding of the derived public key: the low 20 bytes of HashP applied to its uncompressed encoding.

Three properties of Definition 4 are used throughout.

- **Public computability.** Derive takes only public inputs and addition on the curve is public, so *anyone* can compute userDerivedAddr from mpcRootPk, vaultAddr and keyPath. The private half requires $\mathsf{sk}_{\mathsf{root}}$ and therefore exists only inside SigNet. This is what makes a deposit's keyPath $=$ userComm(userSk) sufficient to expose the depositor's EVM address to an observer of $\mathcal{L}_{\mathsf{shd}}$ alone.
- **Contract scoping.** vaultAddr enters the derivation string, so no contract can reach another contract's derived accounts.
- **Path separation.** Within one contract, distinct paths give unrelated keys. The vault uses exactly three: userComm(userSk) for a depositor's account, the constant "vault" for the vault's own pooled account, and the fixed string "midnight response key" for mpcRespKey, which signs attestations and never signs transactions.

The key version keyVersion selects which root key is used and is carried in every request record; it is suppressed above, and throughout the document, since a single version is in use.

Derived Addresses:

The value keyPath can be:

- User address: userDerivedAddr uses keyPath $=$ userComm(userSk) (defined below)
- Vault address: vaultEvmAddr uses keyPath $= \text{''}\texttt{vault}\text{''}$
- MPC Response Address: mpcRespKey uses keyPath $= \text{''}\texttt{midnightresponsekey}\text{''}$. The attestation key mpcRespKey is derived the same way with a fixed protocol string and pinned at `initialise`.

4.2 User's Keys

A user U_i of the cross-chain system holds:

- **Application secret key:** userSk_i: a private key for the cross-chain system: $\mathsf{userSk}_i \leftarrow \{0,1\}^{\lambda}$. This is randomly chosen upon the first Deposit request.
- **Public account** $\mathsf{pubAddr}_i$ on $\mathcal{L}_{\mathsf{pub}}$; (used to fund the derived account).
- **Shielded Account** $(\mathsf{shdAcct}_i, \mathsf{shdAcctSKey}_i)$ on $\mathcal{L}_{\mathsf{shd}}$.
  In Midnight a shielded account consists of two public keys:
  1. **Coin Key-Pair** (Spending Key):
     (a) coinSk = SHA-256$\big($"midnight:csk" $\|$ seed$\big)$
     (b) coinPk $=$ SHA-256$\big($"midnight:zswap-pk[v1]" $\|$ coinSk$\big)$
  2. **Encryption Key-Pair** (Discovery key):
     (a) encSk: a group element generated via Poseidon hash function.

(b) $\text{encPk} = g^{\text{encSk}}$

So

$$\text{shdAcct} = \big((\text{coinSk}, \text{coinPk}),\ (\text{encSk}, \text{encPk})\big)$$

Key	Exposure	Use
coinPk	public	**ownership**: used as a rcpt when Minting
coinSk	secret	**spending**: used as sender when Spending
encPk	public	**payability**: used to compute ciphertext in Spending a coin
encSk	secret	**scanning** : used to decrypt in order to Receive a coin

The circuits of our contract only use the coinPk as a parameter to mint shielded coins. The other keys are used by the Midnight's circuits.

4.2.1 User Authentication Values

We use the following hash functions:

- HashK: A keyed hash function, used to compute our authentication tags. We require that this function satisfies Pseudorandomness (Def. 5) and Multi-user Pseudorandomness (Def. 6) in addition to collision-resistance. Recall also that collision-resistance implies one-wayness (pre-image resistance).

$$\begin{aligned} \text{userComm}(\text{userSk}) &= \text{HashK}(\text{“vault:user:”},\ \text{userSk}) && \text{identity; the MPC path in Deposit.} \\ \text{refundComm}(\text{userSk}, \text{reqId}) &= \text{HashK}(\text{“vault:refund:”},\ \text{userSk},\ \text{reqId}) && \text{the authentication tag.} \end{aligned}$$

  We instantiate HashK with the `transientHash` implemented by Midnight [3](based on Poseidon [15]). Poseidon follows a sponge-like approach and is a good candidate for Multi-user PRF security [10].
- HashP: A collision resistance hash function used to compute digests.

$$\begin{aligned} \text{domSep}(\text{tokType}) &= \text{HashK}(\text{“erc20:vault:”},\ \text{tokType}) && \text{one shielded type per ERC20} \\ \text{reqId} &= \text{HashP}(\text{req}) && \text{over the whole request record (**except skemas**)} \end{aligned}$$

The **outcome** the MPC attests, is computed using the following formula:

$$\text{attDigest}(\text{reqId}, \text{evmOut}) = \text{HashP}(\text{reqId} \,||\, \text{evmOut.kind} || \text{evmOut.length} || \text{evmOut.blockheight})$$

We instantiate HashK with the `transientHash` implemented by Midnight.

4.3 The vault's public state

[3]Lines L75-L81.

Here	Source name	Role
reqMap	`signBidirectionalEventMap`	withdraw and approve requests, keyed by reqId; the MPC reads them here.
depositMap	`depositEventMap`	deposit requests keyed by reqId
swapMap	`swapEventMap`	swap requests keyed by reqId
supplyMap	`supplyEventMap`	supply requests keyed by reqId
redeemMap	`redeemEventMap`	redeem requests keyed by reqId
depositView	`depositSettleViews`	pending-*deposit* record: the depositor commitment, the ERC20 and the amount `completeDeposit` mints
wdView	`withdrawSettleViews`	pending-*withdrawal* record: the refund commitment, the ERC20 and the amount.
swapView	`swapSettleViews`	pending-*swap* record: the refund commitment, tokType, tokType′, amtOut and amtMax.
supplyView	`supplySettleViews`	pending-*supply* record: the refund commitment and the amount.
redeemView	`redeemSettleViews`	pending-*redeem* record: the refund commitment and the share count.
mpcRespKey	`mpcResponseKey`	signature verification key for attestation
reqCounter	`signetRequestNonce`	a counter making otherwise identical requests distinct; its value publicly orders every request the vault has made
initialized	`initialised`	one-shot setup flag
vaultEvmAddr	`vaultEvmAddress`	the contract-enforced recipient of every deposit
routerAddr	`uniswapRouter`	the only spender `approveRouter` approves, and the only destination a swap request sends to
underlyingAddr	`stataUnderlying`	the lent ERC20; gives `startSupply` its burn color and `completeRedeem` its mint color
wrapperAddr	`stataToken`	the interest-bearing wrapper; gives `startRedeem` its burn color and `completeSupply` its mint color. The only spender `approveStata` approves, and the only destination a supply or redeem request sends to

Also pinned at setup and public: the vault's own address vaultAddr, the EVM chain identifier, the Signet signer, and the deployer's identity commitment

Differences with the latest release (erc20-vault-v0.4.0): The latest release replaces reqCounter and the five settle-view maps with a request queue shared by every action: `inputRequestBuffer` and `outputRequestBuffer` hold requests before and after nonce assignment, `inputAttestationBuffer` and `outputAttestationBuffer` hold verified attestations before and after they are folded into the block-height watermark `globalLastSeen`, `vaultAccountNonce` is the vault EVM account's next nonce, and `evictionMap` maps a sent reqId back to its queue entry. Each action keeps a map of its queued arguments (`depositArgsMap`, `withdrawArgsMap`, ...) and a map of its sent requests (`bidirectionalDepositMap`, ...). A queue entry carries exactly one user-dependent value, the ownership commitment $\text{ownComm}(\text{inIndex}, \text{userSk}) = \text{HashK}(\text{userSk} \,\|\, \text{inIndex} \,\|\, \text{“vault:request:”})$ over a caller-chosen random index inIndex, which stands in for $\text{refundComm}(\text{userSk}, \text{reqId})$. Everything else in these tables is bookkeeping (nonces, block heights, argument hashes and attestation digests) that does not depend on the caller, so the tables do not alter the privacy proof. They preserve the property that each request is unique and processed once: completion consumes the entry, and a second identical request is refused while the first is open.

4.4 The shielded Layer: Midnight Computation

Midnight provides shielded coin payments and private circuit evaluation. We highlight with shaded blue the computation that are strictly Midnight's layer and not the vault's own logic.

Midnight shielded-coin layer. In Midnight, a raw secret coin is the tuple: $c = (\text{nonce}, \text{color}, \text{value})$. Here are the functions applied to the secret coin:

$$\begin{aligned} \text{coinCommit}(c, \text{rcpt}) &= \text{PersistantHash}\big(\text{cc-tag} \,\|\, \text{nonce} \,\|\, \text{color} \,\|\, \text{value} \,\|\, \text{rcpt}\big) \\ \text{coinNull}(c, \text{sndr}) &= \text{PersistantHash}\big(\text{cn-tag} \,\|\, \text{nonce} \,\|\, \text{color} \,\|\, \text{value} \,\|\, \text{sndr}\big) \\ \text{tokenTypeOf}(\text{domSep}(\text{tokType}), \text{vaultAddr}) &= \text{PersistantHash}\big(\text{tok-tag} \,\|\, \text{domSep}(\text{tokType}) \,\|\, \text{vaultAddr}\big) \end{aligned}$$

Midnight Private-Circuit Computation. Midnight's contract computation is performed through circuits. Every argument of a circuit is a *private* input of the proof. A call to a circuit C on private arguments in and witness w produces public outputs and state changes for the vault contract Δ_{vault} and the Midnight chain Δ_{coin}; it also produces a statement x and a zk proof π.

$$\big(\ \Delta_{\text{vault}},\ \ \Delta_{\text{coin}},\ \ x,\ \ \pi\ \big).$$

Output	It contains	Chain Updates
Δ_{vault}	the list of assignments the call makes to the vault's public entries of §4.3	**the vault's** state on $\mathcal{L}_{\text{shd}}$
Δ_{coin}	the list of coin commitments, nullifiers and mint records the call declares	**Midnight's** state, not the vault's.
x	the public statement: the tuple an observer of $\mathcal{L}_{\text{shd}}$ reads off the call	—
π	a proof that $R_C(x, w)$ holds	—

Δ_{vault}, Δ_{coin} and x respect the following inclusion:

$$\Delta_{\text{vault}} \subseteq x, \qquad \Delta_{\text{coin}} \subseteq x, \qquad \text{and in general} \qquad x \supsetneq \Delta_{\text{vault}} \cup \Delta_{\text{coin}}.$$

- x also carries values that do not update any state: the identifier reqId, the commitment comCallIO, the attestation the caller presented, and π itself.
- R_C is a *predicate over* x *and* w, not a state update. This statement is verified inside the proof. Everything in x is visible.

Midnight Commitments. We write comCallIO to denote the commitments to the inputs and outputs of the circuit being called which are produced by the circuit computation. The statement x contains comCallIO.

How to read a circuit box. *Arguments* are the inputs to the circuit (private); *witness* are the secret used to prove relations, *reads* are the public entries consulted; *steps* are the assertions and computations in source order; A ■ marked line or a blue block highlights Midnight's functions.

5 OUR IMPLEMENTATION: CIRCUITS

In this section we show our implementation of the interface of Definition 1.

Two calls per flow. Each procedure of Definition 1 is performed by the circuits below. Each action: Deposit, Withdraw, Swap, Supply, Redeem, is performed by *two* calls to the vault contract, with the SigNet bridge in between. The first call records what the MPC should sign and takes the values relevant to the call (transaction parameters). The second presents SigNet's attestation of what happened on $\mathcal{L}_{\text{pub}}$ (Ethereum), and complete the request on $\mathcal{L}_{\text{shd}}$ (Midnight). Request calls are prefixed with start; settlement calls are prefixed with complete The following table maps each call to their circuits.

Procedure	Circuits that implement it
Setup	`initialise` (Circuit C1); `approveRouter` (Circuit C2); `approveStata` (Circuit C12)
Deposit	`startDeposit` (Circuit C3), then `completeDeposit` (Circuit C4)
Withdraw	`startWithdraw` (Circuit C5), then `completeWithdraw` (Circuit C7) or `refundWithdraw` (Circuit C8)
Swap	`startSwap` (Circuit C9), then `completeSwap` (Circuit C10) or `refundSwap` (Circuit C11)
Supply	`startSupply` (Circuit C13), then `completeSupply` (Circuit C14) or `refundSupply` (Circuit C15)
Redeem	`startRedeem` (Circuit C16), then `completeRedeem` (Circuit C17) or `refundRedeem` (Circuit C18)

Differences with the latest release (erc20-vault-v0.4.0): In the latest release the global counter reqCounter is dropped. A request id is made distinct by the EVM transaction nonce instead, and

freshness comes from the block-height watermark `globalLastSeen`. Transaction nonces are not assigned at request time by the caller of `startX` (for any action X except Deposit, whose caller names the nonce of their own derived account), but by a dedicated circuit that any party can call.

Specifically, each circuit `startX` only validates and records the transaction parameters, without the EVM nonce and without the request id, under a caller-chosen random index inIndex. This *partial* request is added to the buffer `inputRequestBuffer` of requests waiting for nonce assignment. Every so often the queue is processed by the circuit `flushQueue`, which assigns the vault account's next nonce `vaultAccountNonce` (a counter only `flushQueue` reads or writes) to each vault-signed request and stamps every request with the current `globalLastSeen`, and by the per-action circuits `sendX` (`sendDeposit`, `sendWithdraw`, ...), which build the complete request from the flushed entry, compute reqId $=$ HashP(req) and notify the MPC. On the way back, the circuits `queueAttestationN` verify the MPC's attestation signature and queue the attestation, `flushQueue` folds it into `outputAttestationBuffer` and raises `globalLastSeen` to its block height, and `completeX` consumes the request together with its attestation. `flushQueue`, `sendX` and `queueAttestationN` are permissionless, take no witness and can be executed by everyone. Note, these circuits only add nonce and time-stamping information that is not user-dependent. As such they do not alter the privacy analysis described in Sec. 7. They implement efficiency optimisations, contention avoidance and replay-attack protection. The refund circuits (`refundX`) are folded into `completeX`, which refunds on a failed or unviable verdict, and every `completeX`, the success branch of `completeWithdraw` included, is gated on the requester's ownership commitment.

Notification to SigNet. Every request call ends with a cross-contract call to the Signet signer, which emits an event the MPC polls. We write notify(vaultAddr, reqId, M) for that event: it carries the vault's address, the request identifier, and the identity of the ledger map M in which the request record was stored, so that the MPC knows where to read req from.

5.1 Setup: $\text{Setup}^{\mathcal{L}_{\text{pub}},\mathcal{L}_{\text{shd}}}(1^\lambda)$

pp $\leftarrow \text{Setup}^{\mathcal{L}_{\text{pub}},\mathcal{L}_{\text{shd}}}(1^\lambda)$

Run once by the deployer, who holds a secret userSk_D of their own.

Step 1. **Deploy the contract** on $\mathcal{L}_{\text{shd}}$ with two constructor arguments: the deployer's identity deployerComm $=$ $\text{userComm}(\text{userSk}_D)$ and the address of the Signet signer. The contract's own address vaultAddr becomes known here, which is why the remaining values cannot be pinned in the constructor.

Step 2. **Derive off-chain**

$$\text{vaultEvmAddr} \leftarrow \text{Derive}(\text{mpcRootPk}, \text{vaultAddr}, \text{“vault”}),$$
$$\text{mpcRespKey} \leftarrow \text{Derive}(\text{mpcRootPk}, \text{vaultAddr}, \text{“midnight response key”}).$$

Step 3. **Run `initialise`** — Circuit C1.

Step 4. **Run `approveRouter`** once per ERC20 that will ever be swapped — Circuit C2.

Output the public parameters:
pp $=$ (vaultAddr, vaultEvmAddr, mpcRespKey, chainId, routerAddr, signer address).

Circuit C1 — `initialise`

Arguments. vaultEvmAddr, routerAddr, underlyingAddr, wrapperAddr, chainId, mpcRespKey. ***Witness.*** userSk. ***Reads.*** initialized, deployerComm. ***Steps.***

1. **assert** initialized $= 0$.
2. **assert** userComm(userSk) $=$ deployerComm.
3. **assert** chainId > 0, routerAddr $\neq 0$, underlyingAddr $\neq 0$, wrapperAddr $\neq 0$.
4. Write initialized $\leftarrow 1$ and the six pinned values.

Output.

Δ_{vault} : initialized $\leftarrow 1$; vaultEvmAddr, routerAddr, underlyingAddr, wrapperAddr, chainId, mpcRespKey written.
Δ_{coin} : $\emptyset$.
x : (`initialise`, Δ_{vault}, comCallIO).
w : userSk.
R : userComm(userSk) $=$ deployerComm.

Circuit C2 — `approveRouter`

Arguments. tokType, evmNonce, keyVersion. ***Witness.*** none. ***Reads.*** initialized, routerAddr, vaultAddr, reqCounter, chainId. ***Steps.***

1. **assert** initialized ≥ 1 and tokType $\neq 0$.
2. evmTx $\leftarrow$ "approve routerAddr for an unlimited amount of tokType". *The spender and the amount come from pinned state; the caller chooses only the token.*
3. req $\leftarrow$ (vaultAddr, reqCounter, keyVersion, keyPath="vault", evmTx, chainId); reqId $\leftarrow$ HashP(req); **assert** reqId $\notin$ reqMap.

Output.

Δ_{vault} : reqCounter $+= 1$; reqMap[reqId] $\leftarrow$ req — *never removed by any circuit*.
Δ_{coin} : $\emptyset$.
x : (`approveRouter`, req, reqId, notify(vaultAddr, reqId, reqMap)).
w : $\perp$.
R : true - the witness is empty and every argument is in x, so π certifies a computation an observer can redo.

5.2 Deposit

$\Pi.\text{Deposit}^{\mathcal{L}_{\text{pub}},\mathcal{L}_{\text{shd}}}$([pubAddr], shdAcct, tokType, amt, aux ; (userSk, mintNonce)).

Private input. secret key userSk; and a fresh mintNonce, used at Step 4.
Public arguments. tokType is an ERC20 contract, and aux $=$ (evmNonce, gasEnvelope, keyVersion). The receiving ZKSwap account shdAcct is not public per-se, but we do not aim to protect it.

Step 1. **Fund the derived account:** outside any circuit, on $\mathcal{L}_{\text{pub}}$. The caller pays amt of tokType to the MPC controlled account userDerivedAddr on $\mathcal{L}_{\text{pub}}$. Where, userDerivedAddr $=$ Derive(mpcRootPk, vaultAddr, userComm(userSk)). See §4.2.1.

Step 2. **Request call:** `startDeposit` (Circuit C3). Recomputes keyPath $\leftarrow$ userComm(userSk) from the

witness and request transfer(vaultEvmAddr, amt) under reqId. $\Delta_{\text{coin}} = \emptyset$*: no coin moves.*

Step 3. **MPC**. The committee signs on behalf of userDerivedAddr a transfer to vaultEvmAddr, and posts attSig over attDigest(reqId, evmOut).

Step 4. **Settlement call:** `completeDeposit` (Circuit C4). Verifies attSig, consumes reqMap[reqId], authenticated the request on userComm(userSk) = req.keyPath, and mints amt to shdAcct using randomness mintNonce.

Outcome: succeeded once Step 4 completes; failed if the EVM transfer did not execute, in which case the value never left userDerivedAddr; Private output: userSk, mintNonce.

Circuit C3 — `startDeposit`

Arguments. evmNonce, gasEnvelope, keyVersion, (tokType, amt). ***Witness.*** userSk. ***Reads.*** initialized, vaultEvmAddr, vaultAddr, reqCounter, chainId. ***Steps.***

1. **assert** initialized ≥ 1, tokType $\neq 0$, gasEnvelope > 0, $0 <$ amt $\leq 2^{64} - 1$.
2. keyPath $\leftarrow$ userComm(userSk).
3. evmTx $\leftarrow$ "transfer(vaultEvmAddr, amt) on tokType", with evmNonce, gasEnvelope, on chainId.
4. req $\leftarrow$ (vaultAddr, reqCounter, keyVersion, keyPath, evmTx, chainId); reqId $\leftarrow$ HashP(req); **assert** reqId $\notin$ depositMap.

Output.

Δ_{vault} : reqCounter $+= 1$; depositMap[reqId] $\leftarrow$ req; depositView[reqId] $\leftarrow$ (keyPath, tokType, amt).
Δ_{coin} : $\emptyset$.
x : (`startDeposit`, req, reqId, depositView[reqId], notify(vaultAddr, reqId, depositMap), comCallIO), where req exposes tokType, amt, vaultEvmAddr, evmNonce, gasEnvelope, reqCounter and keyPath = userComm(userSk).
w : userSk.
R : req.keyPath = userComm(userSk) $\wedge$ depositView[reqId].comm = userComm(userSk).

Circuit C4 — `completeDeposit`

Arguments. reqId, attSig, evmOut, mintNonce, shdAcct. ***Witness.*** userSk. ***Reads.*** initialized, mpcRespKey, depositMap, depositView[reqId]. ***Steps.***

1. **assert** initialized ≥ 1 and evmOut reports success.
2. **assert** EcdsaVerify(attDigest(reqId, evmOut), attSig, mpcRespKey) — **gate 1**, the authentication of the MPC.
3. **assert** reqId $\in$ depositMap; remove depositMap[reqId] — **gate 3**, at most once, enforced by the ledger through Δ_{vault}, not by R.
4. **assert** userComm(userSk) = depositView[reqId].comm; remove depositView[reqId] — **gate 2**, enforces depositor-only.
5. amt $\leftarrow$ depositView[reqId].amt, tokType $\leftarrow$ depositView[reqId].erc20.

Midnight operation. Execute Midnight Circuit `mintShieldedToken` on input domSep, amt, mintNonce, shdAcct. Let newCoin = (mintNonce, color, amt)

where color = tokenTypeOf(domSep(tokType), vaultAddr) ***Output.***

Δ_{vault} : depositMap[reqId] removed; depositView[reqId] removed.
Δ_{coin} : ■ mint record (domSep(tokType), amt); ■ output coinCommit(newCoin, shdAcct).
x : (`completeDeposit`, reqId, attSig, (domSep(tokType), amt), coinCommit(newCoin, shdAcct)).
w : (userSk, mintNonce, shdAcct).
R : userComm(userSk) $=$ depositView[reqId].comm $\wedge$ EcdsaVerify(attDigest(reqId, evmOut), attSig, mpcRespKey) $\wedge$ the published commitment opens to (mintNonce, ·, amt, shdAcct).

Transcript from Deposit

On $\mathcal{L}_{\text{pub}}$: [pubAddr $\rightarrow$ userDerivedAddr], and userDerivedAddr $\rightarrow$ vaultEvmAddr (signed by the committee), both in the clear.

On $\mathcal{L}_{\text{shd}}$: $x_{\texttt{startDeposit}}$ and $x_{\texttt{completeDeposit}}$.

An observer therefore holds userComm(userSk), hence userDerivedAddr, hence the edge from the caller's own EVM account to their identity in this system; tokType and amt; reqId, so **the settlement call is linked to the request call**. Hidden: userSk, mintNonce.

5.3 Withdraw

Withdraw$^{\mathcal{L}_{\text{pub}},\mathcal{L}_{\text{shd}}}$(shdAcct, destAddr$*$, tokType, amt, aux ; privIn)

Private input. privIn holds userSk; the secret key coinSk associated to the $ZKSwap$ account shdAcct; a fresh burnCoinNonce, used at Step 1; and, only if the flow ends in failed, a fresh mintNonce used at Step 4. *Public arguments.* aux $=$ (evmNonce, keyVersion) only: the gas envelope is fixed in the contract, because the *vault* pays for this transaction.

Step 1. **Request call:** `startWithdraw` (Circuit C5). The caller hands the contract a coin descriptor surrCoin $=$ (burnCoinNonce, color, amt); the circuit burns it, records transfer(destAddr*, amt) under reqId with keyPath $=$ "vault" o, and add the authentication tag to the map: wdView[reqId] $\leftarrow$ refundComm(userSk, reqId).

Step 2. **MPC**. The MPC signs the evm request with the key derived at keyPath $=$ "vault", so from vaultEvmAddr, the pool's own account. Once the request is executed, the MPC signs an attestation attSig of the outcome evmOut.

Step 3. **Settlement call on success:** `completeWithdraw`, success branch (Circuit C7). $\Delta_{\text{coin}} = \emptyset$ **and** $w = \perp$ **on this branch**.

Step 4. **Settlement call on failure:** `completeWithdraw`**'s failure branch, or** `refundWithdraw` **-** Circuit C7 and Circuit C8, both which gates on refundComm(userSk, reqId) against the stored marker and re-mints amt under a fresh mintNonce. The two circuits differ only in which attested output routes to them: an executed transfer that returned false goes to `completeWithdraw`; a transaction that never executed goes to `refundWithdraw`.

Outcome: succeeded via Step 3, failed via Step 4. Private output: userSk.

Circuit C5 — `startWithdraw`

Arguments. evmNonce, keyVersion, (tokType, amt, destAddr), and surrCoin $=$ (burnCoinNonce, color, value). ***Witness.*** userSk (and coinSk at the wallet layer). ***Reads.*** initialized, vaultAddr, reqCounter, chainId. ***Steps.***

1. **assert** initialized ≥ 1, tokType $\neq 0$, $0 <$ amt $\leq 2^{64} - 1$.
2. **assert** surrCoin.color $=$ tokenTypeOf(domSep(tokType), vaultAddr) and surrCoin.value $=$ amt.
3. evmTx $\leftarrow$ "transfer(destAddr, amt) on tokType", with evmNonce and a contract-fixed gas envelope, on chainId.
4. keyPath $\leftarrow$ "vault".
5. req $\leftarrow$ (vaultAddr, reqCounter, keyVersion, keyPath, evmTx, chainId); reqId $\leftarrow$ HashP(req); **assert** reqId $\notin$ reqMap.

Midnight operations: burn. `receiveShielded` (surrCoin) then `sendImmediateShielded` (surrCoin, burnAddr, amt).

Output.

Δ_{vault} : reqCounter += 1; reqMap[reqId] $\leftarrow$ req; wdView[reqId] $\leftarrow$ (refundComm(userSk, reqId), tokType, amt).

Δ_{coin} : ■ coinCommit(surrCoin, vaultAddr), ■ coinNull(surrCoin, vaultAddr), ■ coinCommit(burnOutCoin, burnAddr), ■ $\{\text{coinNull}(\text{fundCoin}_i, \text{coinSk})\}_i$ and coinCommit(changeCoin, shdAcct).

x : (`startWithdraw`, req, reqId, wdView[reqId], Δ_{coin}, notify(vaultAddr, reqId, reqMap), comCallIO). The view publishes tokType and amt.

w : (userSk, burnCoinNonce, coinSk).

R : surrCoin.color $=$ tokenTypeOf(domSep(tokType), vaultAddr) $\wedge$ surrCoin.value $=$ amt $\wedge$ wdView[reqId].comm $=$ refundComm(userSk, reqId).

Circuit C6 — failGate - shared across refund circuits (`assertAttestedFailureOutput`)

Not exported. Called first by `refundWithdraw`, `refundSwap`, `refundSupply` and `refundRedeem`. ***Arguments.*** reqId, attSig, evmOut of width 5. ***Reads.*** initialized, mpcRespKey. ***Steps.***

1. **assert** initialized ≥ 1.
2. **assert** EcdsaVerify(attDigest(reqId, evmOut), attSig, mpcRespKey).
3. **assert** evmOut $=$ failed.

Output.

Δ_{vault} : none of its own.

Δ_{coin} : $\emptyset$.

x : (reqId, attSig, evmOut), merged into the caller circuit's statement.

w : $\perp$.

R : EcdsaVerify(attDigest(reqId, evmOut), attSig, mpcRespKey) $\wedge$ evmOut $=$ failed.

Circuit C7 — `completeWithdraw`

Arguments. reqId, attSig, evmOut of width 1, mintNonce. ***Witness.*** userSk - *failure branch only*. ***Reads.*** initialized, mpcRespKey, wdView[reqId], reqMap. ***Steps.***

1. **assert** initialized ≥ 1 and **assert** EcdsaVerify(attDigest(reqId, evmOut), attSig, mpcRespKey).
2. **assert** reqId $\in$ wdView.
3. Remove reqMap[reqId]. The record is not read.
4. ok $\leftarrow$ the success bit of evmOut.
5. If ok $= 0$: **assert** refundComm(userSk, reqId) $=$ wdView[reqId].comm, then mint newCoin $=$

$(\text{mintNonce}, \text{tokenTypeOf}(\text{domSep}(\text{wdView}[\text{reqId}].\text{erc20}), \text{vaultAddr}), \text{wdView}[\text{reqId}].\text{amt})$ to the caller's own shielded account.

6. Remove wdView[reqId].

Output.

Δ_{vault} : reqMap[reqId] removed; wdView[reqId] removed.

Δ_{coin} : ok $= 1$: $\emptyset$. ok $= 0$: ■ mint record $(\text{domSep}(\text{wdView}[\text{reqId}].\text{erc20}), \text{wdView}[\text{reqId}].\text{amt})$; ■ coinCommit(newCoin, caller).

x : $(\texttt{completeWithdraw}, \text{reqId}, \text{attSig}, \text{ok}, \Delta_{\text{coin}})$.

w : ok $= 1$: $\perp$. ok $= 0$: (userSk, mintNonce).

R : ok $= 1$: EcdsaVerify(attDigest(reqId, evmOut), attSig, mpcRespKey). ok $= 0$: the same, $\wedge$ refundComm(userSk, reqId) $=$ wdView[reqId].comm $\wedge$ the published commitment opens to (mintNonce, $\cdot$, wdView[reqId].amt, caller).

$w = \perp$ ***on the success branch.*** The proof certifies a statement with no secret in it, so any third party holding the attestation can produce the same call. mintNonce is an argument on both branches; on the success branch it is simply unused.

Circuit C8 — `refundWithdraw`

Arguments. reqId, attSig, evmOut of width 5, mintNonce. ***Witness.*** userSk. ***Reads.*** initialized, mpcRespKey, wdView[reqId], reqMap. ***Steps.***

1. Run failGate(reqId, attSig, evmOut).
2. **assert** reqId $\in$ wdView.
3. **assert** refundComm(userSk, reqId) $=$ wdView[reqId].comm (authenticating the withdrawer).
4. Remove reqMap[reqId] and wdView[reqId].

Midnight operation. Mint, to the caller's own shielded account,

newCoin $= (\text{mintNonce}, \text{tokenTypeOf}(\text{domSep}(\text{wdView}[\text{reqId}].\text{erc20}), \text{vaultAddr}), \text{wdView}[\text{reqId}].\text{amt})$.

Output.

Δ_{vault} : reqMap[reqId] removed; wdView[reqId] removed.

Δ_{coin} : ■ mint record $(\text{domSep}(\text{wdView}[\text{reqId}].\text{erc20}), \text{wdView}[\text{reqId}].\text{amt})$; ■ coinCommit(newCoin, caller).

x : $(\texttt{refundWithdraw}, \text{reqId}, \text{attSig}, \Delta_{\text{vault}}, \Delta_{\text{coin}})$.

w : (userSk, mintNonce).

R : refundComm(userSk, reqId) $=$ wdView[reqId].comm $\wedge$ the relation of failGate $\wedge$ the published commitment opens to (mintNonce, $\cdot$, wdView[reqId].amt, caller).

What Withdraw **publishes**

On $\mathcal{L}_{\text{pub}}$: on success, one transfer vaultEvmAddr $\rightarrow$ destAddr of amt of tokType. *Nothing tied to* shdAcct *appears.*

On $\mathcal{L}_{\text{shd}}$: $x_{\texttt{startWithdraw}}$, the settlement statement. Of these, the components that involve the caller at all are: the marker refundComm(userSk, reqId); the coins sent to the burn address burnAddr in the

`startWithdraw` (the identity of the payer for this coins is protected by Mindnight's library under the assumption that burnCoinNonce is private and is random), and – in case of failure – the new coins minted to the caller of `refundWithdraw` (the identity of the payer for this coins is protected by Mindnight's library under the assumption that mintNonce is private and is random).

Hidden: which shielded account funded the call/ asked for a refund.

5.4 Swap

$\Pi.\text{Swap}^{\mathcal{L}_{\text{pub}},\mathcal{L}_{\text{shd}}}(\text{shdAcct}, \text{tokType}, \text{amt}, \text{tokType}', \text{amt}', \text{aux}\,;\,\text{privIn})$

Private input. As Withdraw: userSk, coinSk of shdAcct, a fresh burnCoinNonce at Step 1, and a fresh mintNonce at Step 3 and a fresh changeNonce in Step 4.). *Public arguments.* amt is a *maximum* input amtMax and amt′ an exact output amtOut; (in an honest use shdAcct is the recipient of the swapped coin); aux $= (\text{evmNonce}, \text{keyVersion}, \text{feeTier})$.

Step 1. **Request call:** `startSwap` - Circuit C9. Same shape as `startWithdraw`: a coin descriptor for amtMax of tokType is surrendered and burned, an exact-output exchange is recorded under reqId in the map swapMap, and swapView[reqId] $\leftarrow$ refundComm(userSk, reqId) is recorded.

Step 2. **MPC**. As in withdraw, the MPC signs the swap request on behalf of vaultEvmAddr. Once the request is executed, the MPC signs an attestation attSig of the outcome evmOut.

Step 3. **Settlement call on success:** `completeSwap` - Circuit C10. If the transaction is successful, the user can claim the new-colored coin and the change, after proving knowledge of the pre-image of refundComm.

Step 4. **Settlement call on failure:** `refundSwap`. If the transaction reverts on $\mathcal{L}_{\text{pub}}$, user calls Circuit Circuit C11 to re-mint a coin with the full amtMax of the original color, after proving knowledge of the pre-image of refundComm.

Outcome: succeeded via Step 3, failed via Step 4. Private output: the new coins minted.

Circuit C9 — `startSwap`

Arguments. evmNonce, keyVersion, (tokType, tokType′, feeTier, amtOut, amtMax), and surrCoin $=$ (burnCoinNonce, color, value). ***Witness.*** userSk (and coinSk at the wallet layer). ***Reads.*** initialized, routerAddr, vaultEvmAddr, vaultAddr, reqCounter, chainId. ***Steps.***

1. **assert** initialized ≥ 1, tokType $\neq 0$, tokType′ $\neq 0$, $0 <$ amtOut $\leq 2^{64}-1$, $0 <$ amtMax $\leq 2^{64}-1$.
2. **assert** surrCoin.color $=$ tokenTypeOf(domSep(tokType), vaultAddr) and surrCoin.value $=$ amtMax.
3. evmTx $\leftarrow$ "exact-output exchange of at most amtMax of tokType for exactly amtOut of tokType′ at feeTier, recipient is vaultEvmAddr, no price bound", to routerAddr, contract-fixed gas envelope.
4. keyPath $\leftarrow$ "vault", as in `startWithdraw`.
5. req $\leftarrow$ (vaultAddr, reqCounter, keyVersion, keyPath, evmTx, chainId); reqId $\leftarrow$ HashP(req); **assert** reqId $\notin$ swapMap.

Midnight operations: **burn**, exactly as in Circuit C5: `receiveShielded` (surrCoin) then `sendImmediateShielded` (surrCoin, burnAddr, amtMax).

Output.

Δ_{vault} : reqCounter $+= 1$; swapMap[reqId] $\leftarrow$ req; swapView[reqId] $\leftarrow$

$(\text{refundComm}(\text{userSk}, \text{reqId}), \text{tokType}, \text{tokType}', \text{amtOut}, \text{amtMax})$.

Δ_{coin} : as Circuit C5, with amtMax in place of amt: ■ coinCommit(surrCoin, vaultAddr), ■ coinNull(surrCoin, vaultAddr), ■ coinCommit(burnOutCoin, burnAddr), ■ coinNull(fundCoin, coinSk) and coinCommit(changeCoin, shdAcct).

x : $(\texttt{startSwap}, \text{req}, \text{reqId}, \text{swapView}[\text{reqId}], \Delta_{\text{coin}}, \text{notify}(\text{vaultAddr}, \text{reqId}, \text{swapMap}), \text{comCallIO})$, where req exposes tokType, tokType′, feeTier, amtOut, amtMax. The view publishes tokType, tokType′, amtOut and amtMax.

w : (userSk, burnCoinNonce, coinSk).

R : as Circuit C5, over swapView and amtMax.

Circuit C10 — `completeSwap`

Arguments. reqId, attSig, evmOut of width 8, mintNonce, changeNonce. ***Witness.*** userSk, mintNonce, changeNonce. ***Reads.*** initialized, mpcRespKey, swapView[reqId], swapMap. ***Steps.***

1. **assert** initialized ≥ 1; **assert** changeNonce $\neq$ mintNonce.
2. **assert** EcdsaVerify$(\text{attDigest}(\text{reqId}, \text{evmOut}), \text{attSig}, \text{mpcRespKey})$.
3. **assert** reqId $\in$ swapView; remove swapMap[reqId].
4. **assert** refundComm(userSk, reqId) = swapView[reqId].comm (authenticating the swapper); remove swapView[reqId].
5. amtOut $\leftarrow$ swapView[reqId].amtOut, tokType′ $\leftarrow$ swapView[reqId].tokenOut, amtMax $\leftarrow$ swapView[reqId].amtMax, tokType $\leftarrow$ swapView[reqId].tokenIn; amtSpent $\leftarrow$ the attested value in evmOut. amtRemainder $\leftarrow$ amtMax $-$ amtSpent.

Midnight operations: two mints. `mintShieldedToken` is called twice, both times to the caller's own shielded account: newCoin $=$ $(\text{mintNonce}, \text{tokenTypeOf}(\text{domSep}(\text{tokType}'), \text{vaultAddr}), \text{amtOut})$ and remainderCoin $=$ $(\text{changeNonce}, \text{tokenTypeOf}(\text{domSep}(\text{tokType}), \text{vaultAddr}), \text{amtRemainder})$. The second runs even when amtRemainder $= 0$.

Output.

Δ_{vault} : swapMap[reqId] removed; swapView[reqId] removed.

Δ_{coin} : ■ mint record $(\text{domSep}(\text{tokType}'), \text{amtOut})$; ■ mint record $(\text{domSep}(\text{tokType}), \text{amtRemainder})$; ■ coinCommit(newCoin, caller); ■ coinCommit(remainderCoin, caller).

x : $(\texttt{completeSwap}, \text{reqId}, \text{attSig}, \text{amtSpent}, \Delta_{\text{vault}}, \Delta_{\text{coin}})$.

w : (userSk, mintNonce, changeNonce).

R : refundComm(userSk, reqId) $=$ swapView[reqId].comm $\wedge$ EcdsaVerify(attDigest(reqId, evmOut), attSig, mpcRespKey) $\wedge$ changeNonce $\neq$ mintNonce $\wedge$ the first commitment opens to (mintNonce, ·, amtOut, caller) $\wedge$ the second opens to (changeNonce, ·, amtRemainder, caller).

Circuit C11 — `refundSwap`

Arguments. reqId, attSig, evmOut of with 5, mintNonce. ***Witness.*** userSk. ***Reads.*** initialized, mpcRespKey, swapView[reqId], swapMap. ***Steps.***

1. Run failGate(reqId, attSig, evmOut).
2. **assert** reqId $\in$ swapView.
3. **assert** refundComm(userSk, reqId) $=$ swapView[reqId].comm - **the swapper gate**.
4. Remove swapMap[reqId] and swapView[reqId].

Midnight operation. Mint, to the caller's own shielded account, newCoin $=$ $(\text{mintNonce}, \text{color}, \text{swapView}[\text{reqId}].\text{amtMax})$.

where color $=$ $\text{tokenTypeOf}(\text{domSep}(\text{swapView}[\text{reqId}].\text{tokenIn}), \text{vaultAddr})$. ***Output.***

Δ_{vault} : swapMap[reqId] removed; swapView[reqId] removed.

Δ_{coin} : ■ mint record $(\text{domSep}(\text{swapView}[\text{reqId}].\text{tokenIn}), \text{swapView}[\text{reqId}].\text{amtMax})$;
■ $\text{coinCommit}(\text{newCoin}, \text{caller})$.

x : $(\texttt{refundSwap}, \text{reqId}, \text{attSig}, \Delta_{\text{vault}}, \Delta_{\text{coin}})$.

w : $(\text{userSk}, \text{mintNonce})$.

R : $\text{refundComm}(\text{userSk}, \text{reqId}) = \text{swapView}[\text{reqId}].\text{comm} \wedge$ the relation of failGate $\wedge$ the published commitment opens to $(\text{mintNonce}, \cdot, \text{swapView}[\text{reqId}].\text{amtMax}, \text{caller})$.

What Swap **publishes**

On $\mathcal{L}_{\text{pub}}$: the exchange in the clear: both tokens/colors and amounts, feeTier, the recipient vaultEvmAddr, amtMax, amtOut, and the resulting movements through the pool.

On $\mathcal{L}_{\text{shd}}$: as Withdraw, over swapMap and swapView, with two mint records at settlement. Mints do not leak any information about the payer and receiver.

The executed price is public. amtMax $\in$ req and amtSpent $\in x_{\texttt{completeSwap}}$ outright, and amtChange is published as a mint record as well. With both tokens, feeTier and the block position also public,every vault swap is a fully transparent trade. No address - besides the vault address - is involved in the trade.

5.5 Supply **and** Redeem

The lending flow Supply and Redeem moves value between an ERC20 and its interest-bearing wrapper, both pinned at Setup (§4.3), the token being lent is at address underlyingAddr, the receipt token is at address wrapperAddr. Supply and Redeem are both a Swap. Supply burns a coin of the underlying color and mints one of the wrapper color; Redeem does the reverse. They differ from Swap in that **the minted amount is read from the attestation, not from the request**. In Swap, amtOut is a field of req that the caller fixed before anything was burned; in Supply and Redeem the corresponding quantity is whatever the lending market returned, and it reaches the circuit only through evmOut.

Throughout the paper we use:

$$\begin{aligned} \text{color}_{\text{u}} &= \text{tokenTypeOf}(\text{domSep}(\text{underlyingAddr}), \text{vaultAddr}), \\ \text{color}_{\text{w}} &= \text{tokenTypeOf}(\text{domSep}(\text{wrapperAddr}), \text{vaultAddr}) \end{aligned}$$

for the two shielded colors involved: color_{u} the color of the lent ERC20, color_{w} the color of the wrapper.

5.5.1 Supply

$\Pi.\text{Supply}^{\mathcal{L}_{\text{pub}}, \mathcal{L}_{\text{shd}}}(\text{shdAcct}, \text{tokType}, \text{amt}, \text{aux}\,;\, \text{privIn})$

Private input. privIn holds userSk; the secret key coinSk associated to the ZKSwap account shdAcct; a fresh burnCoinNonce, used at Step 1; and a fresh mintNonce, used at Step 3 on success or at Step 4 on failure.
Public arguments. tokType $=$ underlyingAddr, read from pinned state rather than supplied; aux $=$ $(\text{evmNonce}, \text{keyVersion})$ only. The gas envelope is contract-fixed, because the *vault*'s own EVM account

pays for this transaction. Requires `approveStata` (Circuit C12) to have been run once, so that the wrapper may pull the underlying.

Step 1. **Request call:** `startSupply` (Circuit C13). The caller hands the contract a coin descriptor surrCoin $=$ (burnCoinNonce, color_u, amt); the circuit burns it, files "deposit(amt, vaultEvmAddr) on wrapperAddr" under reqId in supplyMap with keyPath $=$ "vault", and pins supplyView[reqId] $\leftarrow$ refundComm(userSk, reqId). The caller address does not appear in the transaction.

Step 2. **MPC.** The committee signs with the key derived at keyPath $=$ "vault", so from vaultEvmAddr, the pool's own account; the relayer broadcasts; the committee observes the execution, reads the returned share count against schemaIn, re-serialises it against schemaOut, and posts attSig over attDigest(reqId, evmOut).

Step 3. **Settlement call on success:** `completeSupply` (Circuit C14). Supplier-gated; mints *one* coin, of color color_w and value amtShares read from evmOut.

Step 4. **Settlement call on failure: execute** `refundSupply` (Circuit C15): Re-mints the *full* amt of color color_u - the deposit never executed, so nothing was spent.

Outcome: succeeded via Step 3, failed via Step 4. Private output: userSk, and the new coins minted.

Circuit C12 — `approveStata`

Arguments. evmNonce, keyVersion. ***Witness.*** none. ***Reads.*** initialized, underlyingAddr, wrapperAddr, vaultAddr, reqCounter, chainId. ***Steps.***

1. **assert** initialized ≥ 1.
2. evmTx $\leftarrow$ "approve wrapperAddr for an unlimited amount", sent to underlyingAddr, contract-fixed gas envelope.
3. keyPath $\leftarrow$ "vault"; req $\leftarrow$ (vaultAddr, reqCounter, keyVersion, keyPath, evmTx, chainId); reqId $\leftarrow$ HashP(req); **assert** reqId $\notin$ reqMap.

Output.

Δ_{vault} : reqCounter $+= 1$; reqMap[reqId] $\leftarrow$ req.
Δ_{coin} : $\emptyset$.
x : (`approveStata`, req, reqId, notify(vaultAddr, reqId, reqMap), comCallIO).
w : $\perp$.
R : the request is well formed.

Circuit C13 — `startSupply` : `startSupply`

Arguments. evmNonce, keyVersion, amt, and surrCoin $=$ (burnCoinNonce, color, value). ***Witness.*** userSk (and coinSk at the wallet layer). ***Reads.*** initialized, underlyingAddr, wrapperAddr, vaultEvmAddr, vaultAddr, reqCounter, chainId. ***Steps.***

1. **assert** initialized ≥ 1, $0 <$ amt $\leq 2^{64} - 1$.
2. **assert** surrCoin.color $= \text{color}_u$ and surrCoin.value $=$ amt.
3. evmTx $\leftarrow$ "`deposit(uint256,address)` for amt, recipient vaultEvmAddr", sent to wrapperAddr, contract-fixed gas envelope.
4. keyPath $\leftarrow$ "vault".
5. req $\leftarrow$ (vaultAddr, reqCounter, keyVersion, keyPath, evmTx, chainId); reqId $\leftarrow$ HashP(req); **assert** reqId $\notin$ supplyMap.

Midnight operations: burn. `receiveShielded` (surrCoin) then `sendImmediateShielded` (surrCoin, burnAddr, amt).

Output.

Δ_{vault} : reqCounter $+=$ 1; supplyMap[reqId] $\leftarrow$ req; supplyView[reqId] $\leftarrow$ (refundComm(userSk, reqId), amt).

Δ_{coin} : ■ coinCommit(surrCoin, vaultAddr), ■ coinNull(surrCoin, vaultAddr), ■ coinCommit(burnOutCoin, burnAddr), ■ {coinNull(fundCoin, coinSk)} and coinCommit(changeCoin, shdAcct).

x : (`startSupply`, req, reqId, supplyView[reqId], Δ_{coin}, notify(vaultAddr, reqId, supplyMap), comCallIO).

w : (userSk, burnCoinNonce, coinSk).

R : surrCoin.color $=$ color_{u} $\wedge$ surrCoin.value $=$ amt $\wedge$ supplyView[reqId].comm $=$ refundComm(userSk, reqId).

Circuit C14 — `completeSupply`

Arguments. reqId, attSig, evmOut of width 8, mintNonce. ***Witness.*** userSk, mintNonce. ***Reads.*** initialized, mpcRespKey, supplyView[reqId], supplyMap, wrapperAddr. ***Steps.***

1. **assert** initialized ≥ 1 and **assert** EcdsaVerify(attDigest(reqId, evmOut), attSig, mpcRespKey).
2. **assert** reqId $\in$ supplyMap; remove supplyMap[reqId].
3. **assert** refundComm(userSk, reqId) $=$ supplyView[reqId].comm (authentication **supplier-only**); remove supplyView[reqId].
4. amtShares $\leftarrow$ the attested value in evmOut, released publicly.

Midnight operation. Mint newCoinShares $=$ (mintNonce, color_{w}, amtShares) to user(coinPk of setlAcct).

Output.

Δ_{vault} : supplyMap[reqId] removed; supplyView[reqId] removed.

Δ_{coin} : ■ mint record (domSep(wrapperAddr), amtShares); ■ coinCommit(newCoinShares, coinPk of shdAcct).

x : (`completeSupply`, reqId, attSig, amtShares, Δ_{vault}, Δ_{coin}).

w : (userSk, mintNonce).

R : refundComm(userSk, reqId) $=$ supplyView[reqId].comm $\wedge$ EcdsaVerify(attDigest(reqId, evmOut), attSig, mpcRespKey) $\wedge$ the commitment opens to (mintNonce, color_{w}, amtShares, coinPk).

Circuit C15 — `refundSupply`

Arguments. reqId, attSig, evmOut of with 5, mintNonce. ***Witness.*** userSk. ***Reads.*** initialized, mpcRespKey, supplyView[reqId], supplyMap. ***Steps.***

1. Run failGate(reqId, attSig, evmOut).
2. **assert** reqId $\in$ supplyView.
3. **assert** refundComm(userSk, reqId) $=$ supplyView[reqId].comm - (authentication: **supplier-only**).
4. Remove supplyMap[reqId] and supplyView[reqId].

Midnight operation. Mint newCoin $=$ $(\text{mintNonce}, \text{color}_u, \text{supplyView}[\text{reqId}].\text{amt})$ to $\text{user}(\text{coinPk of setlAcct})$.

Output.

Δ_{vault} : supplyMap[reqId] removed; supplyView[reqId] removed.

Δ_{coin} : ■ mint record $(\text{domSep}(\text{underlyingAddr}), \text{supplyView}[\text{reqId}].\text{amt})$;
■ $\text{coinCommit}(\text{newCoin}, \text{user}(\text{coinPk of setlAcct}))$.

x : $(\texttt{refundSupply}, \text{reqId}, \text{attSig}, \Delta_{\text{vault}}, \Delta_{\text{coin}})$.

w : $(\text{userSk}, \text{mintNonce})$.

R : $\text{refundComm}(\text{userSk}, \text{reqId}) = \text{supplyView}[\text{reqId}].\text{comm} \wedge$ the relation of failGate $\wedge$ the commitment opens to $(\text{mintNonce}, \text{color}_u, \text{supplyView}[\text{reqId}].\text{amt}, \text{user}(\text{coinPk of shdAcct}))$.

What Supply **publishes**

On $\mathcal{L}_{\text{pub}}$: the supply in the clear: the wrapper contract called, the amount of the underlying supplied, the receiver vaultEvmAddr, and the share count returned. The vault's own account is the sender, so *nothing tied to* shdAcct *appears*.

On $\mathcal{L}_{\text{shd}}$: $x_{\texttt{startSupply}}$, the MPC signatures, and the settlement statement. Of these, the components that involve the caller at all are: the marker $\text{refundComm}(\text{userSk}, \text{reqId})$; the coins burned to the contract: the nullifier and the new coin commitment destined to burnAddr (the private and random nonce burnCoinNonce protects the identity of the caller of `startSupply`), the new private coins minted in the `completeSupply`/`refundSupply` destined to shdAcct (the private and random nonce mintNonce protects the identity of the recipient).

The exchange rate is public. amt is a field of req and amtShares is released by the settlement call and published again as a mint record, so amt/amtShares. This is information about $\mathcal{L}_{\text{pub}}$, not about the caller, and it is identical for every supplier in the same block.

Hidden: which shielded account funded the call; which shielded account received the shares.

5.5.2 Redeem

$\Pi.\text{Redeem}^{\mathcal{L}_{\text{pub}}, \mathcal{L}_{\text{shd}}}(\text{shdAcct}, \text{wrap}(\text{tokType}), \text{amtShares}, \text{aux}\,;\, \text{privIn})$

Private input. As Supply: userSk; coinSk of shdAcct; a fresh burnCoinNonce at Step 1; and a fresh mintNonce at Step 3 on success or Step 4 on failure.
Public arguments. $\text{wrap}(\text{tokType}) = \text{wrapperAddr}$, read from pinned state; $\text{aux} = (\text{evmNonce}, \text{keyVersion})$ only. No approval leg is needed: the vault redeems its own shares, so no allowance to a third party is involved.

Step 1. **Request call:** `startRedeem` (Circuit C16). The caller hands the contract a coin descriptor $\text{surrCoin} = (\text{burnCoinNonce}, \text{color}_w, \text{amtShares})$; the circuit burns it, files "redeem(amtShares, vaultEvmAddr, vaultEvmAddr) on wrapperAddr" under reqId in redeemMap with keyPath = "vault", and pins $\text{redeemView}[\text{reqId}] \leftarrow \text{refundComm}(\text{userSk}, \text{reqId})$.

Step 2. **MPC.** As in Supply, signing from vaultEvmAddr. The committee reports amtAssets, the amount of the underlying the market returned — principal plus accrued interest.

Step 3. **Settlement call on success:** `completeRedeem` (Circuit C17). Redeemer-gated; mints *one* coin, of colour color_u and value amtAssets.

Step 4. **Settlement call on failure:** `refundRedeem` (Circuit C18). Re-mints the *full* amtShares of color color_w.

Outcome: succeeded via Step 3, failed via Step 4, pending otherwise. Private output: userSk.

Circuit C16 — `startRedeem` : (`startRedeem`)

Arguments. evmNonce, keyVersion, amtShares, and surrCoin $=$ (burnCoinNonce, color, value). ***Witness.*** userSk (and coinSk at the wallet layer). ***Reads.*** initialized, wrapperAddr, vaultEvmAddr, vaultAddr, reqCounter, chainId. ***Steps.***

1. **assert** initialized ≥ 1, $0 <$ amtShares $\leq 2^{64} - 1$.
2. **assert** surrCoin.color $=$ color$_w$ and surrCoin.value $=$ amtShares.
3. evmTx $\leftarrow$ "`redeem(uint256,address,address)` for amtShares, receiver vaultEvmAddr, owner vaultEvmAddr", sent to wrapperAddr, contract-fixed gas envelope.
4. keyPath $\leftarrow$ "vault".
5. req $\leftarrow$ (vaultAddr, reqCounter, keyVersion, keyPath, evmTx, chainId); reqId $\leftarrow$ HashP(req); **assert** reqId $\notin$ redeemMap.

Midnight operations: burn. `receiveShielded` (surrCoin) then `sendImmediateShielded` (surrCoin, burnAddr, amtShare

Output.

Δ_{vault} : reqCounter $+=$ 1; redeemMap[reqId] $\leftarrow$ req; redeemView[reqId] $\leftarrow$ (refundComm(userSk, reqId), amtShares).

Δ_{coin} : ■ coinCommit(surrCoin, vaultAddr), ■ coinNull(surrCoin, vaultAddr), ■ coinCommit(burnOutCoin, burnAddr), ■ $\{\text{coinNull}(\text{fundCoin}_i, \text{coinSk})\}_i$ and coinCommit(changeCoin, shdAcct).

x : (`startRedeem`, req, reqId, redeemView[reqId], Δ_{coin}, notify(vaultAddr, reqId, redeemMap), comCallIO).

w : (userSk, burnCoinNonce, coinSk).

R : surrCoin.color $=$ color$_w$ $\wedge$ surrCoin.value $=$ amtShares $\wedge$ redeemView[reqId].comm $=$ refundComm(userSk, reqId).

Circuit C17 — `completeRedeem`

Arguments. reqId, attSig, evmOut of width 8, mintNonce. ***Witness.*** userSk, mintNonce. ***Reads.*** initialized, mpcRespKey, redeemView[reqId], redeemMap, underlyingAddr. ***Steps.***

1. **assert** initialized ≥ 1 and **assert** EcdsaVerify(attDigest(reqId, evmOut), attSig, mpcRespKey).
2. **assert** reqId $\in$ redeemMap; remove redeemMap[reqId].
3. **assert** refundComm(userSk, reqId) $=$ redeemView[reqId].comm — **redeemer-only**; remove redeemView[reqId].
4. amtAssets $\leftarrow$ the attested value in evmOut, **released publicly**.

Midnight operation. Mint newCoinAssets $=$ (mintNonce, color$_u$, amtAssets) to coinPk.

Output.

Δ_{vault} : redeemMap[reqId] removed; redeemView[reqId] removed.

Δ_{coin} : ■ mint record (domSep(underlyingAddr), amtAssets); ■ coinCommit(newCoinAssets, coinPk).

x : (`completeRedeem`, reqId, attSig, amtAssets, Δ_{vault}, Δ_{coin}).

w : (userSk, mintNonce).

R: refundComm(userSk, reqId) = redeemView[reqId].comm $\wedge$ EcdsaVerify(attDigest(reqId, evmOut), attSig, mpcRespKey) $\wedge$ the commitment opens to (mintNonce, color_u, amtAssets, coinPk).

Circuit C18 — `refundRedeem`

Arguments. reqId, attSig, evmOut of width 5, mintNonce. ***Witness.*** userSk. ***Reads.*** initialized, mpcRespKey, redeemView[reqId], redeemMap. ***Steps.***

1. Run failGate(reqId, attSig, evmOut).
2. **assert** reqId $\in$ redeemView.
3. **assert** refundComm(userSk, reqId) = redeemView[reqId].comm (authenticating **redeemer-only**).
4. Remove redeemMap[reqId] and redeemView[reqId].

Midnight operation. Mint newCoin = (mintNonce, color_w, redeemView[reqId].shares) to user(coinPk of setlAcct).

Output.

Δ_{vault}: redeemMap[reqId] removed; redeemView[reqId] removed.
Δ_{coin}: ■ mint record (domSep(wrapperAddr), redeemView[reqId].shares); ■ coinCommit(newCoin, user(coinPk of setlAcct)).
x: (`refundRedeem`, reqId, attSig, Δ_{vault}, Δ_{coin}).
w: (userSk, mintNonce).
R: refundComm(userSk, reqId) = redeemView[reqId].comm $\wedge$ the relation of failGate $\wedge$ the commitment opens to (mintNonce, color_w, redeemView[reqId].shares, user(coinPk of setlAcct)).

What Redeem **publishes**

On $\mathcal{L}_{\text{pub}}$: the redemption in the clear: the wrapper called, the share count burned, and the amount of the underlying returned. The vault's account vaultEvmAddr is involved in all transactions on $\mathcal{L}_{\text{pub}}$ so nothing tied to shdAcct appears.

On $\mathcal{L}_{\text{shd}}$: as Supply, over redeemMap and redeemView, with the two colors exchanged.

The accrued yield is public. amtShares is a field of req and amtAssets is released by the settlement call, so the realised rate is recoverable, and with it the interest the position earned.

Hidden: which shielded account funded the call; which received the assets.

6 CRYPTOGRAPHIC ASSUMPTIONS

Before presenting a formal security proof for the unlikability (§ 7) and soundness (§ 8) properties of the protocol described in Section 5, we define the cryptographic assumptions that we use in the proofs. Most of the cryptographic assumptions we list below are inherited from the Midnight shielded-circuit and shielded-coin layer..

Notation λ is the security parameter; PPT means probabilistic polynomial time in λ; $\mu(\cdot)$ denotes a negligible function. $x \leftarrow \mathcal{D}$ is sampling, $y \leftarrow F(x)$ the output of a randomized algorithm, $\mathcal{A}^{\mathcal{O}}$ an adversary

with access to the oracle set $\mathcal{O}$. Notation $p_a \approx_\mu p_b$ means that probability distribution p_a is equal to p_a up to a negligible factor.

6.1 (Multi-use) Pseudorandom functions

Let $\mathsf{F} : \mathcal{K} \times \mathcal{X} \to \mathcal{Y}$ be a family of functions.

Definition 5 (PRF security)**.** For a PPT adversary $\mathcal{A}$ define

$$\mathsf{Adv}^{\mathsf{prf}}_{\mathsf{F}}(\mathcal{A}, \lambda) \;=\; \Big|\, \mathsf{Pr}\big[\, \mathsf{k} \leftarrow \mathcal{K} \;:\; \mathcal{A}^{\mathsf{F}(\mathsf{k},\cdot)}(1^\lambda) = 1 \,\big] \;-\; \mathsf{Pr}\big[\, \mathsf{RF} \leftarrow \mathsf{Funs}(\mathcal{X}, \mathcal{Y}) \;:\; \mathcal{A}^{\mathsf{RF}(\cdot)}(1^\lambda) = 1 \,\big] \,\Big|,$$

where $\mathsf{Funs}(\mathcal{X}, \mathcal{Y})$ is the set of all functions from $\mathcal{X}$ to $\mathcal{Y}$. F is a *pseudorandom function* if $\mathsf{Adv}^{\mathsf{prf}}_{\mathsf{F}}(\mathcal{A}, \lambda) \leq \mu(\lambda)$ for every PPT $\mathcal{A}$.

Definition 6 (Multi-user PRF Security [4, 2])**.** For $m = m(\lambda)$ users and a PPT adversary $\mathcal{A}$ with access to m oracles, define

$$\mathsf{Adv}^{\mathsf{mu\text{-}prf}}_{\mathsf{F},m}(\mathcal{A}, \lambda) \;=\; \Big|\, \mathsf{Pr}\big[\, \mathsf{k}_1, \ldots, \mathsf{k}_m \leftarrow \mathcal{K} \;:\; \mathcal{A}^{\mathsf{F}(\mathsf{k}_1,\cdot),\ldots,\mathsf{F}(\mathsf{k}_m,\cdot)}(1^\lambda) = 1 \,\big]$$

$$-\; \mathsf{Pr}\big[\, \mathsf{RF}_1, \ldots, \mathsf{RF}_m \leftarrow \mathsf{Funs}(\mathcal{X}, \mathcal{Y}) \;:\; \mathcal{A}^{\mathsf{RF}_1(\cdot),\ldots,\mathsf{RF}_m(\cdot)}(1^\lambda) = 1 \,\big] \,\Big|.$$

F is *multi-user secure* if $\mathsf{Adv}^{\mathsf{mu\text{-}prf}}_{\mathsf{F},m}(\mathcal{A}, \lambda) \leq \mu(\lambda)$ for every PPT $\mathcal{A}$ and every polynomial m.

Remark 3 (On the need of multi-user PRF security)**.** This property is not strictly needed, as any an hybrid over the m users reduces any multi-input secure PRF (Definition 6) into a secure PRF (Definition 5) with a loss of a factor m. However, the multi-user version is preferred to avoid this loss.

6.2 Hash Functions

Definition 7 (Collision-Resistant Hash Function [18])**.** A hash function family $\mathcal{H} = \{H_{pp} : \{0,1\}^* \to \{0,1\}^{\ell(\lambda)}\}_{pp \in \{0,1\}^\lambda}$ parameterized by public parameters pp and security parameter λ is *collision-resistant* if for every PPT adversary $\mathcal{A}$, there exists a negligible function $\mu(\lambda)$ such that:

$$\mathsf{Pr}\left[pp \leftarrow \mathsf{Gen}(1^\lambda),\ (x, x') \leftarrow \mathcal{A}(1^\lambda, pp) : x \neq x' \wedge H_{pp}(x) = H_{pp}(x')\right] \leq \mu(\lambda)$$

6.3 Non-interactive zero-knowledge arguments

The Midnight layer produces, for every circuit call, a non-interactive argument for the relation R_C. We use the standard notion of a preprocessing non-interactive argument system in the common-reference-string model [7, 16].

Definition 8 (Non-interactive argument system)**.** Let R be an NP relation with statements x and witnesses w. A triple $\Pi_{\mathsf{zk}} = (\mathsf{ZK.Setup}, \mathsf{ZK.Prove}, \mathsf{ZK.Verify})$ of PPT algorithms, where $\mathsf{crs} \leftarrow \mathsf{ZK.Setup}(1^\lambda, R)$, $\pi \leftarrow \mathsf{ZK.Prove}(\mathsf{crs}, x, w)$ and $\mathsf{ZK.Verify}(\mathsf{crs}, x, \pi) \in \{0,1\}$, is a *non-interactive argument system* for R if it satisfies completeness, computational soundness and zero knowledge as defined below.

Definition 9 (Completeness)**.** For every $(x, w) \in R$, $\mathsf{Pr}[\mathsf{crs} \leftarrow \mathsf{ZK.Setup}(1^\lambda, R);\ \pi \leftarrow \mathsf{ZK.Prove}(\mathsf{crs}, x, w) : \mathsf{ZK.Verify}(\mathsf{crs}, x, \pi) = 1] = 1$.

Definition 10 (Computational soundness). For every PPT $\mathcal{A}$,

$$\mathsf{Adv}^{\mathsf{snd}}_{\Pi_{\mathsf{zk}}}(\mathcal{A},\lambda) \;=\; \Pr\big[\,\mathsf{crs}\leftarrow \mathsf{ZK.Setup}(1^\lambda,R);\,(x,\pi)\leftarrow\mathcal{A}(\mathsf{crs})\;:\;\mathsf{ZK.Verify}(\mathsf{crs},x,\pi)=1\wedge x\notin\mathcal{L}_R\,\big]\;\le\;\mu(\lambda),$$

where $\mathcal{L}_R=\{x:\exists w \text{ s.t. } R(x,w)=1\}$.

Definition 11 (Knowledge soundness). Π_{zk} is *knowledge sound* if for every PPT $\mathcal{A}$ there is a PPT extractor $\mathcal{E}$ such that

$$\Pr\big[\,\mathsf{crs}\leftarrow \mathsf{ZK.Setup}(1^\lambda,R);\,(x,\pi)\leftarrow\mathcal{A}(\mathsf{crs});\,w\leftarrow\mathcal{E}^{\mathcal{A}}(\mathsf{crs},x)\;:\;\mathsf{ZK.Verify}(\mathsf{crs},x,\pi)=1\wedge R(x,w)=0\,\big]\le\mu(\lambda).$$

Knowledge soundness implies Definition 10.

Definition 12 (Zero knowledge). Π_{zk} is *(computationally) zero knowledge* if there is a PPT simulator $\mathsf{ZK.Sim}=(\mathsf{ZK.Sim}_1,\mathsf{ZK.Sim}_2)$ such that for every PPT $\mathcal{A}$,

$$\begin{aligned}\mathsf{Adv}^{\mathsf{zk}}_{\Pi_{\mathsf{zk}}}(\mathcal{A},\lambda) \;=\; \Big|&\Pr\big[\mathsf{crs}\leftarrow\mathsf{ZK.Setup}(1^\lambda,R):\mathcal{A}^{\mathcal{P}(\mathsf{crs},\cdot,\cdot)}(\mathsf{crs})=1\big]\\ &-\;\Pr\big[(\mathsf{crs},\mathsf{td})\leftarrow\mathsf{ZK.Sim}_1(1^\lambda,R):\mathcal{A}^{\mathcal{S}(\mathsf{crs},\mathsf{td},\cdot,\cdot)}(\mathsf{crs})=1\big]\Big|\end{aligned}$$

is negligible, where on a query (x,w) with $R(x,w)=1$ the oracle $\mathcal{P}$ returns $\mathsf{ZK.Prove}(\mathsf{crs},x,w)$ and $\mathcal{S}$ returns $\mathsf{ZK.Sim}_2(\mathsf{crs},\mathsf{td},x)$, that is, the simulated proof is produced **from the statement alone**, without the witness. Both oracles return $\perp$ when $R(x,w)=0$.

6.4 Commitment schemes

Definition 13 (Commitment scheme). A *commitment scheme* $\mathsf{CS}=(\mathsf{CS.Gen},\mathsf{CS.Com})$ consists of parameter-generation function $\mathsf{ck}\leftarrow\mathsf{CS.Gen}(1^\lambda)$, taking the security parameter λ and outputting a public key ck and a commitment function $\mathsf{CS.Com}(\mathsf{ck},m;r)$ taking the parameters ck a message m and randomness $r\leftarrow\{0,1\}^\lambda$, and outputting a commitment.

Definition 14 (Computational hiding). CS is *computationally hiding* if for every PPT $\mathcal{A}$,

$$\begin{aligned}\mathsf{Adv}^{\mathsf{hide}}_{\mathsf{CS}}(\mathcal{A},\lambda)=\Big|&\Pr\big[\mathsf{ck}\leftarrow\mathsf{CS.Gen}(1^\lambda);\,(m_0,m_1,\mathsf{st})\leftarrow\mathcal{A}(\mathsf{ck});\,r\leftarrow\{0,1\}^\lambda\;:\;\mathcal{A}(\mathsf{st},\mathsf{CS.Com}(\mathsf{ck},m_0;r))=1\big]\\ &-\Pr\big[\mathsf{ck}\leftarrow\mathsf{CS.Gen}(1^\lambda);\,(m_0,m_1,\mathsf{st})\leftarrow\mathcal{A}(\mathsf{ck});\,r\leftarrow\{0,1\}^\lambda\;:\;\mathcal{A}(\mathsf{st},\mathsf{CS.Com}(\mathsf{ck},m_1;r))=1\big]\Big|\;\le\;\mu(\lambda).\end{aligned}$$

Definition 15 (Computational binding). CS is *computationally binding* if for every PPT $\mathcal{A}$,

$$\begin{aligned}\mathsf{Adv}^{\mathsf{bind}}_{\mathsf{CS}}(\mathcal{A},\lambda) \;=\; \Pr\Big[&\mathsf{ck}\leftarrow\mathsf{CS.Gen}(1^\lambda);\,(m,r,m',r')\leftarrow\mathcal{A}(\mathsf{ck}):\\ &m\neq m'\;\wedge\;\mathsf{CS.Com}(\mathsf{ck},m;r)=\mathsf{CS.Com}(\mathsf{ck},m';r')\Big]\;\le\;\mu(\lambda).\end{aligned}$$

The coin commitment coinCommit of §4.2.1 is used as a commitment in this sense, with the coin nonce playing the role of r and the tuple $(\mathsf{color},\mathsf{value},\mathsf{rcpt})$ the role of m. in [18].

6.5 Message authentication codes

Definition 16 (MAC and existential unforgeability). A *message authentication code* is a pair $\mathsf{MAC}=(\mathsf{Tag},\mathsf{Vrfy})$ with $t\leftarrow\mathsf{Tag}(\mathsf{k},m)$ and $\mathsf{Vrfy}(\mathsf{k},m,t)\in\{0,1\}$, correct in the usual sense. MAC is *existentially unforgeable under chosen-message attack* if for every PPT $\mathcal{A}$,

$$\mathsf{Adv}^{\mathsf{euf\text{-}cma}}_{\mathsf{MAC}}(\mathcal{A},\lambda)=\Pr\big[\mathsf{k}\leftarrow\mathcal{K};\;(m^*,t^*)\leftarrow\mathcal{A}^{\mathsf{Tag}(\mathsf{k},\cdot)}(1^\lambda)\;:\;\mathsf{Vrfy}(\mathsf{k},m^*,t^*)=1\;\wedge\;m^*\notin Q\big]\le\mu(\lambda),$$

where Q is the set of messages $\mathcal{A}$ queried. A pseudorandom function (Definition 5) with $\mathsf{Tag}(\mathsf{k},m)=\mathsf{F}(\mathsf{k},m)$ and verification by recomputation is such a MAC [18].

6.6 Key-Private Public Key Encryption

Definition 17. [Key-Private Encryption / IK-CPA [3]] Let $\Pi = (\mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec})$ be a public-key encryption scheme. We say Π is key-private under chosen-plaintext attack (IK-CPA) if for all probabilistic polynomial-time (PPT) adversaries $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$, the advantage

$$\mathbf{Adv}^{\text{ik-cpa}}_{\Pi,\mathcal{A}}(\lambda) = \left|\Pr\left[\mathbf{Exp}^{\text{ik-cpa-1}}_{\Pi,\mathcal{A}}(\lambda) = 1\right] - \Pr\left[\mathbf{Exp}^{\text{ik-cpa-0}}_{\Pi,\mathcal{A}}(\lambda) = 1\right]\right|$$

is negligible in the security parameter λ, where the experiment is defined as follow:

$\mathbf{Exp}^{\text{ik-cpa-}b}_{\Pi,\mathcal{A}}(\lambda)$ for $b \in \{0,1\}$

1. $(pk_0, sk_0) \leftarrow \mathsf{KeyGen}(1^\lambda)$ and $(pk_1, sk_1) \leftarrow \mathsf{KeyGen}(1^\lambda)$
2. $(m, \mathsf{st}) \leftarrow \mathcal{A}_1(pk_0, pk_1)$
3. $c \leftarrow \mathsf{Enc}(pk_b, m)$
4. $b' \leftarrow \mathcal{A}_2(c, \mathsf{st})$
5. Return b'

6.7 Honest Verifier ZK Proofs

Definition 18. [Honest-Verifier Zero-Knowledge [14]] Let (P, V) be an interactive proof system for a language L associated with an **NP**-relation $R(x, w)$. Let $\mathsf{View}_V\langle P(w), V\rangle(x)$ denote the random variable representing V's view during an execution of $\langle P(w), V\rangle$ on input x with witness w. We say (P, V) is Computational Honest-Verifier Zero-Knowledge (c-HVZK) if there exists a PPT simulator $\mathcal{S}$ such that:

$$\{\mathcal{S}(x)\}_{x\in L} \overset{c}{\approx} \{\mathsf{View}_V\langle P(w), V\rangle(x)\}_{(x,w)\in R}$$

where $\overset{c}{\approx}$ denotes computational indistinguishability over sequences indexed by $x \in L$.

6.8 Signatures with key derivation and pre-signatures

Definition 19 (EUF-CMA under additive key derivation and pre-signatures)**.** Let SIG = $(\mathsf{SIG.KGen}, \mathsf{SIG.Sign}, \mathsf{SIG.Vrfy})$ be a signature scheme with a public key-derivation function Derive mapping a master public key and a derivation path to a derived public key. SIG is *unforgeable under additive key derivation with pre-signatures* if for every PPT $\mathcal{A}$ that receives the master public key, may request signatures under any derived key on messages of its choice, and may additionally request pre-signatures (first-round signing material generated before the message is known),

$$\mathsf{Adv}^{\text{euf-cma}}_{\text{akd}}(\mathcal{A}, \lambda) = \Pr\big[\,\mathcal{A} \text{ outputs } (\mathsf{keyPath}^*, m^*, \sigma^*) \text{ such that}$$
$$\mathsf{SIG.Vrfy}\big(\mathsf{Derive}(\mathsf{mpcRootPk}, \mathsf{vaultAddr}, \mathsf{keyPath}^*),\, m^*,\, \sigma^*\big) = 1 \text{ and } m^* \text{ was never queried at path } \mathsf{keyPath}^*\,\big]$$

is negligible. The security of ECDSA in this model – additive key derivation together with pre-signatures, is analyzed in [17].

7 PROOF OF WITHDRAWER UNLINKABILITY

Differences with the latest release (erc20-vault-v0.4.0): The same proof *strategy* described in this section applies to the latest release. This is because the circuits it adds (`flushQueue`, `sendX`,

`queueAttestationN`) are simply **bookkeeping** circuits that label the requests differently. This labeling **does not add any user-dependent information** to the requests, as such they do not introduce any link to the caller. Furthermore the circuits are *permissionless* – they can be called by anyone, without needing proof of any secret and *anonymous* – the identity of the caller is not revealed at application layer. Two changes to the circuits this section does analyse must be carried through the proof: the marker refundComm$(\text{userSk}, \text{reqId})$ becomes ownComm$(\text{inIndex}, \text{userSk})$, a keyed hash of the same secret over the caller-chosen random index inIndex in place of reqId (the source requires inIndex to be fresh per request, which the argument must assume), and `completeWithdraw` proves that marker on every verdict, so its success branch is no longer witness-free.

7.1 Proof Overview

As we discussed in Section 2, withdrawer unlinkability informally says the following: a cross-chain protocol is unlinkable if, for any users U_0, U_1, with shielded accounts, $\text{shAcct}_0, \text{shAcct}_1$ who are *equally likely to have called a withdraw* at time t for amount x, and given the *transcript of all messages exchanged up to that point*[4], no one should be able to infer if the withdrawer was shAcct_0 or shAcct_1 (in polynomial time). The same definition can be extended to every action (i.e., Swap, Supply, Redeem), except Deposit.

In this section, we provide a formal proof that protocol Π (Section 5) achieves withdrawer unlinkability § 2, assuming cryptographic assumptions stated in § 6 and assuming the privacy and soundness of the Midnight shielded-coin and shielded-computation layer.

Proof Intuition and Approach. The intuition of the proof is simple: to prove that our protocol achieves unlinkability, we need to prove that the protocol messages generated upon execution of any action call (Withdraw, Swap, Supply, Redeem) – except Deposit – do not leak any information about the identity of the caller.

The approach to provide such proof is to analyze the *transcript* of each call: look at every message generated by our protocol, identify what messages *encode the identity of the caller*, and then use the security assumption of the cryptographic tools employed, to conclude that such encoding hides the identity from any polynomial time adversary.

7.2 Intuition for the Proof of Withdrawer-unlinkability.

A withdraw request consists of two parts: (1) the transaction that will be executed on the public chain (2) the transactions executed on the private chain. Let us call shdAcct_i, the user who is executing the Withdraw flow (i.e., `startWithdraw`, `completeWithdraw`).

Transcript on public chain $\mathcal{L}_{\text{pub}}$ (Ethereum). The transaction executed on the public chain req contains the sender vaultEvmAddr, the recipient of the withdraw destAddr* and the amount. The sender is a fixed constant vaultEvmAddr, independent of the caller shdAcct_i. Hence, *the request on the public chain does not contain any information about the identity of the withdrawer.* **Note. If a user re-uses existing evm address for** destAddr **instead of a fresh one, our protocol provides no unlinkability guarantees.**

[4] In the actual definition the adversary see the transcript of all messages exchanged before and after that particular withdraw request, but the actions after the withdraw are normalized to the event that both shAcct_0 and shAcct_1 would have performed that withdraw.

Transcript on private chain $\mathcal{L}_{\text{shd}}$ (Midnight). When a user shdAcct_i of $\mathcal{L}_{\text{shd}}$ (Midnight) starts a withdraw of amt token, it will first create a private payment to a public burn address burnAddr. We use Midnight libraries `receiveShielded` (surrCoin) then `sendImmediateShielded` (surrCoin, burnAddr, amt). Under the assumption that the randomness included in surrCoin is secret and unpredictable to the adversary, the Midnight shielded-coin layer guarantee that the transcript of the above calls does not leak any information about the identity of the payer (shdAcct_i). In addition to the coins generated from the Midnight-coin layer, a withdraw call, also generates an **authentication tag** $\text{refundComm}(\text{userSk}_i, \text{reqId})$, where userSk_i is the application key associate to the user who is calling withdraw. This authentication tag is computed using a pseudorandom function HashK. By the definition (Def 5, Def 6), the outputs of HashK do not reveal any information about the secret key userSk_i used for the evaluation. Yet, HashK is a deterministic function, so, if we call $\text{refundComm}(\text{userSk}_i, \text{reqId})$ on the same userSk_i and reuse an reqId in more than one withdraw request, then the two requests are readily linked. This is why, our implementation relies on the fact that there is a one-to-one mapping between a requestid reqId and a request. Hence, due to the **pseudorandomness** of HashK and the **uniqueness** of reqId, $\text{refundComm}(\text{userSk}_i, \text{reqId})$ does not leak any information about the caller. Finally, if a withdraw request fails, the caller must obtain a refund from the Midnight chain. This is done by `mintShieldedToken` (newCoin). Assuming that newCoin contains a fresh random nonce (mintNonce) that remains private, the privacy property of Midnight shielded-coin layer guarantees that the recipient of the refund is protected. In addition, every execution above is attested by a zero-knowledge proof, via the Midnight private-computation layer (Kachina). Using the privacy guarantees of the latter, we can assume that the the transcript generated by the zk proofs, does not leak any information about the secret used in the execution of every circuit above.

Transcript of other actions (i.e., Swap, Supply, Redeem). The transcript of other actions, all follow a similar structure as Withdraw, and use the same cryptographic assumptions as withdraw. Hence we informally state that these additional transacripts do not leak any information about the caller of a withdrawal. Formally, in the proof this is implicitly obtained in our hybrid-argument proofs. In our hybrid experiments, in every experiment we remove a secret from the transcript, and the change affects every call (not just withdrawal).

7.3 Assumption on Consolidated Coins in Midnight Wallets (External to Our protocol)

Our proof of security (see hybrid Hyb_3 in the formal proof), uses an assumption about user's wallet spending convention. In the proof we assume that **a user will always keep the coins consolidated in one coin**.

Let us explain. At very high level, a coin on Midnight is a tuple (nonce, type, value); creating it publishes a commitment, spending it publishes a nullifier. Spending a coin on Midnight involves (1) publishing the nullifiers (derived from the sender's secret key); (2) creating a new commitment for the *recipient*'s coin, and (3) attaching a zero-knowledge proof of validity of (1) and (2). Nullifiers do not reveal anything about the amount and the spender (to a PPT Adversary), however the *number* of nullifiers used to make a shielded payment reveal *how many coins* were spent. This information could help identifying who is the user who is spending those coins. So, for instance, if an adversary somehow knows that Alice has 3 coins and Bob 2 (even if he does not know the value of them), by looking at the number of nullifiers released in a payment transaction, the adversary could determine if Alice is paying.

The above power is captured by our security game, where we give the adversary the power to choose which actions and transactions honest parties perform as part of her side channel information (See Section 2). So, an adversary could instruct Alice to receive payments of amount 2, and then 3; and

Bob to receive one payment of 5. If Alice does not always keep her coins consolidated in one, when Alice will spend 5, she will use two nullifers (while Bob will use only 1). The number of coins leak the fact that Alice is paying. Therefore, in this security proof **we assume that all honest users keep their wallet consolidated**. This means, at all time, each user keeps **one private coin** (shielded UTXO). And when it spends any amount of that coin, she always mints a change coin.

7.4 Formal Proof

Theorem 1. *If* HashK *satisfies Multi-user PRF Security (Definition 6), Midnight's layer commitments satisfy Hiding (Definition 14), Midnight's circuit outputs satisfy zero-knowledge (Definition 8), and Midnight's nullifiers satisfy Pseudorandomness (Definition 5), and assuming that wallet satisfies the consolidated-coin assumption (Section 7.3), then system Π described in Section 5 achieves withdrawer unlinkability (Definition 2).*

Proof. Our objective is to prove that any PPT adversary $\mathcal{A}$

$$\Pr[\text{Exp-Unlink}_{\Pi,\mathcal{A}}(1^\lambda) = 1] \leq \frac{1}{2} + \mu(\lambda)$$

with Exp-Unlink described in Fig. 1. We prove this statement with a standard hybrid arguments. We start with Hyb_0 corresponding to the real world, where experiment $\text{Exp-Unlink}_{\Pi,\mathcal{A}}$ is executed with the real protocol Π (described in § 5). We start by assuming that $\mathcal{A}$ wins the real-world game $\text{Exp-Unlink}_{\text{Hyb}_0,\mathcal{A}}$ with probability p_0. Then, our goal is to reach a *simulated* world where the challenger is executing a protocol that is completely simulated – no identities/secrets are used anywhere – and claim that $\mathcal{A}$ wins in this simulated world with the "same" probability p_0 (negligibly close). Which allows us to conclude that p_0 must be close to $1/2$ since in the simulated world the protocol messages contain no secret information, and no adversary can guess which account was picked in the game with better than $1/2$ probability.

The core of the proof is to reach the *simulated world* through a sequence of hybrid worlds (or hybrid experiments), $\text{Hyb}_1, \text{Hyb}_2, \ldots$ where we run the game $\text{Exp-Unlink}_{\text{Hyb}_i,\mathcal{A}}$ with a modified protocol Π_{Hyb_i}. In each Π_{Hyb_i}, the protocol is modified by removing the secrets used to compute a cryptographic object. In each hybrid world Hyb_i, we use the cryptographic properties to claim that the probability of $\mathcal{A}$ winning the game in this world must be close to the previous world Hyb_{i-1}, up to a negligible factor. In the last hybrid game Hyb_6, we reach the *simulated* world, a world where no secret is used in any computation. Since secrets are not used, in such a world, the protocol messages do not leak any information about the caller – only the public information about the transactions do. *Since the public information do not contain the identity of the caller of a withdraw, a swap, a supply or a redeem,*, the transcript overall does not leak any information. In this simulated world, the probability that the adversary $\mathcal{A}$ wins the game no better than just guessing. Hence $p_6 = 1/2$, and the sequence of hybrid games allows use to conclude that $p_0 \approx_\mu p_1 \approx \ldots \approx_\mu 1/2$ and therefore $p_0 \approx_\mu 1/2$, where $\approx_\mu$ means equal up to a negligible factor. We provide the formal hybrid-argument proof below.

7.4.1 Hybrid 0

This hybrid corresponds o the game $\text{Exp-Unlink}_{\Pi,\mathcal{A}}(1^\lambda)$ described in Figure 3.2. This is the *real world*, where every oracle is implemented faithfully following protocol Π. When participating in the real-world experiment, $\mathcal{A}$ obtains the following transcript.

■ **Transcript from the oracle calls pre-challenge**: The adversary has oracle access to both ledger and to cross-chain oracles (as explained in Section 2).

$$(\text{shAcct}_0,\ \text{shAcct}_1,\ \text{tokType},\ \text{amt},\ \text{aux}^*,\ \text{outcome}^*,\ \text{st}) \leftarrow \mathcal{A}^{\mathcal{O}}(\text{pp}),$$

As such $\mathcal{A}$ can request creations of honest accounts on $\mathcal{L}_{\text{pub}}$ and $\mathcal{L}_{\text{shd}}$, and instruct actions on behalf of those users with the cross-chain oracles: deposit, withdraw, swap, supply, redeem. First, recall that a user participating in the cross-chain system holds the following keys (these are privet to the user (challenger) and not known to the adversary).:

- **Application secret key:** userSk_i: a private key for the cross-chain system: $\text{userSk}_i \leftarrow \{0,1\}^\lambda$. This is randomly chosen upon the first Deposit request.
- Shielded Account (shdAcct_i, shdAcctSKey_i) on $\mathcal{L}_{\text{shd}}$; (ZKSwap wallet on Midnight).
- Public account pubAddr_i on $\mathcal{L}_{\text{pub}}$; (used to fund the derived account).

The secret keys of the simulated users are held by the challenger who simulates the experiment.

For convenience of the reader below we report an abridged version of the the transcripts that $\mathcal{A}$ would obtain from the interactions with the cross-chain oracles $\mathcal{O}$Deposit, $\mathcal{O}$Withdraw, $\mathcal{O}$Swap, $\mathcal{O}$Supply, $\mathcal{O}$Redeem.
For each oracle call, we highlight:

- Δ_{vault}: messages posted on the contract state on the Midnight chain.
- Δ_{coin}: message posted on the shielded-coin layer on the Midnight chain.
- statements, zk proofs and relations proved by the Midnight private-circuit layer.
- $\mathcal{O}\text{Deposit}(\text{pubAddr}_i, \text{shdAcct}_i, \text{tokType}, \text{amt}, \text{aux}, \text{outcome})$:

Transcript for the Deposit flow

Circuit `startDeposit` (see Circuit C3) ***Output.***

Δ_{vault} : reqCounter $+= 1$; depositMap[reqId] $\leftarrow$ req; depositView[reqId] $\leftarrow$ (userComm(userSk$_i$), tokType, amt).

Δ_{coin} : $\emptyset$.

x : (`startDeposit`, req, reqId, depositView[reqId], notify(vaultAddr, reqId, depositMap), comCallIO), where

$$\text{req} = \text{tokType}, \text{amt}, \text{vaultEvmAddr}, \text{evmNonce}, \text{gasEnvelope}, \text{reqCounter}, \text{keyPath}$$

and $\text{keyPath}_i = \text{userComm}(\text{userSk}_i)$.

zkproofs : ■

R : req.keyPath $=$ userComm(userSk$_i$) $\wedge$ depositView[reqId].comm $=$ userComm(userSk$_i$).

Circuit: `completeDeposit` (see Circuit C4) ***Output.***

Δ_{vault} : depositMap[reqId] removed; depositView[reqId] removed.

Δ_{coin} : ■

x : (`completeDeposit`, reqId, depositView[reqId], attSig, tokType, amt, newMintedCoin) where newMintedCoin $=$ coinCommit(mintNonce, tokType, amt, shdAcct_i).

zkproofs : ■

R : userComm(userSk$_i$) $=$ depositView[reqId].comm $\wedge$ EcdsaVerify(attSig, mpcRespKey) $\wedge$ and newMintedCoin $=$ coinCommit(mintNonce, tokType, amt, shdAcct).

w : (userSk_i, mintNonce, shdAcct)

Note:

We expect that users will re-use the same application secret userSk_i, and hence the same keyPath_i and userDerivedAddr_i for every Deposit call.

– $\mathcal{O}\text{Withdraw}(\text{shdAcct}_i, \text{pubAddr}_j, \text{tokType}, \text{amt}, \text{aux}, \text{outcome})$

Transcript for the Withdraw flow

`startWithdraw` (see Figure C5) ***Output.***

Δ_{vault} : reqCounter $+= 1$; reqMap[reqId] $\leftarrow$ req;
wdView[reqId] $\leftarrow$ (refundComm(userSk$_i$, reqId), tokType, amt).

Δ_{coin} : *Nullifiers published* (one coin spent):

- ■ coinNull(fundCoin, user(coinSk$_i$)) (we assume consolidated-coin): keyed by the caller's **secret**;
- ■ coinNull(surrCoin, contract(vaultAddr)): the contract spending what it has just received, keyed by a **public** address.

Commitments published (one coin created):

- ■ coinCommit(surrCoin, contract(vaultAddr)) : the wallet's caller paying the contract;
- ■ coinCommit(burnOutCoin, burnAddr), over (evolveNonce(burnCoinNonce), color*, amt): the contract paying the burn address;
- ■ coinCommit(changeCoin, user(coinPk)), of value $(V - \text{amt})$, recipient is the caller's public coin key.

x : (`startWithdraw`, req, reqId, wdView[reqId], Δ_{coin}, notify(vaultAddr, reqId, reqMap), comCallIO), where req exposes tokType, amt, destAddr, evmNonce, reqCounter and keyPath $=$ "vault".

R : wdView[reqId].comm $=$ refundComm(userSk, reqId) $\wedge$ surrCoin $=$ (burnCoinNonce, color*, amt) opens both coinCommit(surrCoin, contract(vaultAddr)) and coinNull(surrCoin, contract(vaultAddr)) $\wedge$ each fundCoin opens its published nullifier under coinSk and is unspent $\wedge$ the color balances: $V +$ amt in, amt $+ (V -$ amt$)$ + amt out.

`completeWithdraw` (See circuit C7) ***Output.***

Δ_{vault} : reqMap[reqId] removed; wdView[reqId] removed.

Δ_{coin} : ok $= 1$: $\emptyset$. ok $= 0$: ■ mint record (domSep(wdView[reqId].erc20), wdView[reqId].amt);
■ coinCommit(mintNonce, wdView[reqId].erc20, wdView[reqId].amt, shdAcct$_i$).

x : (`completeWithdraw`, reqId, attSig, ok, wdView[reqId], Δ_{coin}).

R : ok $= 1$: EcdsaVerify(attDigest(reqId, evmOut), attSig, mpcRespKey). ok $= 0$: the same, $\wedge$ refundComm(userSk$_i$, reqId) $=$ wdView[reqId].comm $\wedge$ commitment opening (mintNonce, $\cdot$, wdView[reqId].amt, shdAcct$_i$).

`refundWithdraw` (See circuit C8) ***Output.***

Δ_{vault} : reqMap[reqId] removed; wdView[reqId] removed.

Δ_{coin} : ■ mint record (domSep(wdView[reqId].erc20), wdView[reqId].amt);
■ coinCommit(mintNonce, tokType, amt, shdAcct$_i$).

x : (`refundWithdraw`, reqId, attSig, wdView[reqId], Δ_{vault}, Δ_{coin}).

R : refundComm(userSk, reqId) $=$ wdView[reqId].comm $\wedge$ the relation of failGate $\wedge$ the commitment opens to (mintNonce, $\cdot$, wdView[reqId].amt, shdAcct$_i$).

– $\mathcal{O}\text{Swap}(\text{shdAcct}, \text{tokType}, \text{amt}, \text{tokType}', \text{amt}', \text{aux}, \text{outcome})$

Transcript for the Swap flow.

Circuit `startSwap` (see Table C9) ***Output.***

Δ_{vault} : reqCounter $+= 1$; swapMap[reqId] $\leftarrow$ req; swapView[reqId] $\leftarrow$ (refundComm(userSk, reqId), tokType, tokType′, amtOut, amtMax).

Δ_{coin} : as Circuit C5, with amtMax in place of amt: ■ coinCommit(surrCoin, vaultAddr), ■ coinNull(surrCoin, vaultAddr), ■ coinCommit(burnOutCoin, burnAddr), ■ {coinNull(fundCoin_i, coinSk_i)} and coinCommit(changeCoin, shdAcct_i).

x : (`startSwap`, req, reqId, swapView[reqId], Δ_{coin}, notify(vaultAddr, reqId, swapMap), comCallIO), where req exposes tokType, tokType′, feeTier, amtOut, amtMax.

w : (userSk, burnCoinNonce, coinSk).

R : as Circuit C5, over swapView and amtMax.

Circuit `completeSwap` (see Table C10) ***Output.***

Δ_{vault} : swapMap[reqId] removed; swapView[reqId] removed.

Δ_{coin} : ■ mint records (domSep(tokType′), amtOut) and (domSep(tokType), amtChange); ■ coinCommit(mintNonce, shdAcct_i) and coinCommit(changeNonce, shdAcct_i).

x : (`completeSwap`, reqId, attSig, amtSpent, swapView[reqId], Δ_{coin}).

R : refundComm(userSk, reqId) $=$ swapView[reqId].comm $\wedge$ the ECDSA statement $\wedge$ changeNonce $\neq$ mintNonce $\wedge$ amtSpent $\leq$ amtMax $\wedge$ the two commitments open under mintNonce and changeNonce.

Circuit `refundSwap` (see Table C11) ***Output.***

Δ_{vault} : swapMap[reqId] removed; swapView[reqId] removed.

Δ_{coin} : ■ mint record (domSep(swapView[reqId].tokenIn), swapView[reqId].amtMax); ■ coinCommit(mintNonce, tokType, amtMax, shdAcct_i).

x : (`refundSwap`, reqId, attSig, swapView[reqId], Δ_{vault}, Δ_{coin}).

R : refundComm(userSk, reqId) $=$ swapView[reqId].comm $\wedge$ the relation of failGate $\wedge$ the commitment opens to (mintNonce, ·, swapView[reqId].amtMax, shdAcct_i).

– $\mathcal{O}$Supply(shdAcct, tokType, amt, aux, outcome)

Transcript for the Supply flow.

Circuit `startSupply` (see Table C13) ***Output.***

Δ_{vault} : reqCounter $+= 1$; supplyMap[reqId] $\leftarrow$ req; supplyView[reqId] $\leftarrow$ (refundComm(userSk_i, reqId), amt).

Δ_{coin} : as Circuit C5, with color_u in place of color*: ■ coinCommit(surrCoin, vaultAddr), ■ coinNull(surrCoin, vaultAddr), ■ coinCommit(burnOutCoin, burnAddr), ■ {coinNull(fundCoin_i, coinSk_i)} and coinCommit(changeCoin, shdAcct_i).

x : (`startSupply`, req, reqId, supplyView[reqId], Δ_{coin}, notify(vaultAddr, reqId, supplyMap), comCallIO), where req exposes amt, and underlyingAddr, wrapperAddr, vaultEvmAddr, evmNonce, reqCounter and keyPath $=$ "vault". The notification **names** supplyMap, so it alone identifies the request as a supply.

w : (userSk, burnCoinNonce, coinSk).

R : as Circuit C5, over supplyView and color_u.

Circuit `completeSupply` (see Table C14) ***Output.***

Δ_{vault} : supplyMap[reqId] removed; supplyView[reqId] removed.

Δ_{coin} : ■ mint record (domSep(wrapperAddr), amtShares); ■ coinCommit(mintNonce, wrapperAddr, amtShares, s

x : (`completeSupply`, reqId, attSig, amtShares, supplyView[reqId], Δ_{vault}, Δ_{coin}), where amtShares is read from evmOut.

R: refundComm(userSk, reqId) $=$ supplyView[reqId].comm $\wedge$ the ECDSA statement $\wedge$ the commitment opens to (mintNonce, color_w, amtShares, shdAcct_i).

Circuit `refundSupply` (see Table C15) ***Output.***

Δ_{vault}: supplyMap[reqId] removed; supplyView[reqId] removed.

Δ_{coin}: ■ mint record (domSep(underlyingAddr), supplyView[reqId].amt); ■ coinCommit(mintNonce, underlyingAddr, amt, shdAcct_i).

x: (`refundSupply`, reqId, attSig, supplyView[reqId], Δ_{vault}, Δ_{coin}).

R: refundComm(userSk, reqId) $=$ supplyView[reqId].comm $\wedge$ the relation of failGate $\wedge$ the commitment opens to (mintNonce, color_u, supplyView[reqId].amt, shdAcct_i).

– $\mathcal{O}$Redeem(shdAcct, wrap(tokType), amtShares, aux, outcome)

Transcript for the Redeem flow.

Circuit `startRedeem` (see Table C16) ***Output.***

Δ_{vault}: reqCounter $+=$ 1; redeemMap[reqId] $\leftarrow$ req; redeemView[reqId] $\leftarrow$ (refundComm(userSk_i, reqId), amtShares).

Δ_{coin}: as Circuit C13, with color_w in place of color_u and amtShares in place of amt: ■ coinCommit(surrCoin, vaultAddr), ■ coinNull(surrCoin, vaultAddr), ■ coinCommit(burnOutCoin, burnAddr), ■ {coinNull(fundCoin_i, coinSk_i)} and coinCommit(changeCoin, shdAcct_i).

x: (`startRedeem`, req, reqId, redeemView[reqId], Δ_{coin}, notify(vaultAddr, reqId, redeemMap), comCallIO), where req exposes amtShares and wrapperAddr, vaultEvmAddr, evmNonce, reqCounter and keyPath $=$ "vault". The notification **names** redeemMap.

w: (userSk, burnCoinNonce, coinSk).

R: as Circuit C5, over redeemView and color_w.

Circuit `completeRedeem` (see Table C17) ***Output.***

Δ_{vault}: redeemMap[reqId] removed; redeemView[reqId] removed.

Δ_{coin}: ■ mint record (domSep(underlyingAddr), amtAssets); ■ coinCommit(mintNonce, underlyingAddr, amtAsse

x: (`completeRedeem`, reqId, attSig, amtAssets, redeemView[reqId], Δ_{vault}, Δ_{coin}), where amtAssets is read from evmOut.

R: refundComm(userSk, reqId) $=$ redeemView[reqId].comm $\wedge$ the ECDSA statement $\wedge$ the commitment opens to (mintNonce, color_u, amtAssets, shdAcct_i).

Circuit `refundRedeem` (see Table C18) ***Output.***

Δ_{vault}: redeemMap[reqId] removed; redeemView[reqId] removed.

Δ_{coin}: ■ mint record (domSep(wrapperAddr), redeemView[reqId].shares); ■ coinCommit(mintNonce, wrapperAddr, amtShares, shdAcct_i).

x: (`refundRedeem`, reqId, attSig, redeemView[reqId], Δ_{vault}, Δ_{coin}).

R: refundComm(userSk, reqId) $=$ redeemView[reqId].comm $\wedge$ the relation of failGate $\wedge$ the commitment opens to (mintNonce, color_w, redeemView[reqId].shares, shdAcct_i).

■ **Transcript from the challenge phase:**

In the challenge phase, the adversary $\mathcal{A}$ selects two honest shielded accounts shAcct_0, shAcct_1 and an amount amt to be withdrawn.

The challenge chooses one of the two users at random (shAcct_b), and executes the withdraw call honestly using that account. Namely:

$$\text{Withdraw}(\text{shAcct}_b, \text{destAddr}^*, \text{tokType}, \text{amt}, \text{aux}\,;\, \text{userSk}_b)$$

where destAddr^* is a fresh address created on the public chain.

As result of this call, the adversary will observe the following transcript

Circuit C19 — Transcript for the Withdraw **flow for user** shAcct_b

`startWithdraw` ***Output.***

Δ_{vault} : reqCounter $+= 1$; reqMap[reqId] $\leftarrow$ req; wdView[reqId] $\leftarrow$ $(\text{refundComm}(\text{userSk}_b, \text{reqId}), \text{tokType}, \text{amt})$.

Δ_{coin} : *Nullifiers published* (exactly one coin spent.):

- ■ $\text{coinNull}(\text{fundCoin}, \text{user}(\text{coinSk}_b))$, keyed by the caller's **secret**;
- ■ $\text{coinNull}(\text{surrCoin}, \text{contract}(\text{vaultAddr}))$.

Commitments published (one coin created):

- ■ $\text{coinCommit}(\text{surrCoin}, \text{contract}(\text{vaultAddr}))$ - the wallet paying the contract; **public address again**;
- ■ $\text{coinCommit}(\text{burnOutCoin}, \text{burnAddr})$, over $(\text{evolveNonce}(\text{burnCoinNonce}), \text{color}^*, \text{amt})$ — the contract paying the burn address;
- ■ $\text{coinCommit}(\text{changeCoin}, \text{user}(\text{coinPk}_b))$, of value $V - \text{amt}$ - keyed by the caller's public coin key.

x : $(\texttt{startWithdraw}, \text{req}, \text{reqId}, \text{wdView}[\text{reqId}], \Delta_{\text{coin}}, \text{notify}(\text{vaultAddr}, \text{reqId}, \text{reqMap}), \text{comCallIO})$, where req exposes tokType, amt, destAddr*, evmNonce, reqCounter and keyPath $=$ "vault".

R : wdView[reqId].comm $=$ $\text{refundComm}(\text{shAcct}_b, \text{reqId})$ $\wedge$ surrCoin $=$ $(\text{burnCoinNonce}, \text{color}^*, \text{amt})$ opens both $\text{coinCommit}(\text{surrCoin}, \text{contract}(\text{vaultAddr}))$ and $\text{coinNull}(\text{surrCoin}, \text{contract}(\text{vaultAddr}))$ $\wedge$ fundCoin_i opens its published nullifier under coinSk and is unspent $\wedge$ the color balances: $V +$ amt in, amt $+ (V -$ amt$)$ + amt out.

`completeWithdraw` (See circuit C7) ***Output.***

Δ_{vault} : reqMap[reqId] removed; wdView[reqId] removed.

Δ_{coin} : ok $= 1$: $\emptyset$. ok $= 0$: ■ mint record $(\text{domSep}(\text{wdView}[\text{reqId}].\text{erc20}), \text{wdView}[\text{reqId}].\text{amt})$; ■ $\text{coinCommit}(\text{mintNonce}, \text{tokType}, \text{amt}, \text{shAcct}_b)$.

x : $(\texttt{completeWithdraw}, \text{reqId}, \text{attSig}, \text{ok}, \text{wdView}[\text{reqId}], \Delta_{\text{coin}})$. $(\text{shAcct}_b, \text{mintNonce})$.

R : ok $= 1$: $\text{EcdsaVerify}(\text{attDigest}(\text{reqId}, \text{evmOut}), \text{attSig}, \text{mpcRespKey})$. ok $= 0$: the same, $\wedge$ $\text{refundComm}(\text{shAcct}_b, \text{reqId})$ $=$ wdView[reqId].comm $\wedge$ commitment opening $(\text{mintNonce}, \cdot, \text{wdView}[\text{reqId}].\text{amt}, \text{shAcct}_b)$.

`refundWithdraw` (See circuit C8) ***Output.***

Δ_{vault} : reqMap[reqId] removed; wdView[reqId] removed.

Δ_{coin} : ■ mint record $(\text{domSep}(\text{wdView}[\text{reqId}].\text{erc20}), \text{wdView}[\text{reqId}].\text{amt})$; ■ $\text{coinCommit}(\text{newCoin}, \text{shAcct}_b)$.

x : $(\texttt{refundWithdraw}, \text{reqId}, \text{attSig}, \text{wdView}[\text{reqId}], \Delta_{\text{vault}}, \Delta_{\text{coin}})$.

R : $\text{refundComm}(\text{shAcct}_b, \text{reqId}) = \text{wdView}[\text{reqId}].\text{comm}$ $\wedge$ the relation of failGate $\wedge$ the commitment opens to $(\text{mintNonce}, \cdot, \text{wdView}[\text{reqId}].\text{amt}, \text{shAcct}_b)$.

■ **transcript from the oracle calls post-challenge:** The transcript here is the same as in the pre-challenge phase under the condition specified in the definition (See Definition 2).

Let p_0 be the probability that any PPT $\mathcal{A}$ wins the game $\text{Exp-Unlink}_{\text{Hyb}_0,\mathcal{A}}$, i.e.

$$Pr[\text{Exp-Unlink}_{\text{Hyb}_0,\mathcal{A}} \rightarrow 1] = p_0$$

7.4.2 Hybrid 1 - Midnight Shielded Computation and Shielded Coin Layer and

In this hybrid experiment, we use the properties offered by the Midnight Layer. We consider a sequence of sub-hybrid, and in each one of them we replace the secrets used in a Midnight component, and use

the underlying security of the Mindight layer to claim that our replacement is sound. While we state precisely which assumptions we are using, we do not provide formal reductions here, since it requires to prove the security of the Midnight components, which we assume to be secure by hypothesis assumption.

In Hybrid 1 we will use the following cryptographic propoerties that are implicit in the Midnight shielded comptuation layer.

1. **ZK Proofs protect the secrets (Zero-Knowledge Property 8).** The zk proofs associated to the statements x and relation R satisfy the *zero-knowledge property*. Technically this means that we can replace every proof generated using the user's secrets: userSk_i, coinSk_i, mintNonce, shdAcct_i etc, with a simulated proof that is calculated without any secret.
2. **Commitments protect the receivers 's of the coins (Hiding 14).** In circuits `completeDeposit`, `completeWithdraw`, `completeSwap` coins are calculated as the output of `mintShieldedToken`, using the recipient's identity (shdAcct_i 's public key coin coinPk_i):

$$\text{coinCommit}(c, \text{rcpt}) = \text{PersistantHash}\big(\text{cc-tag} \,\|\, \text{nonce} \,\|\, \text{color} \,\|\, \text{value} \,\|\, \text{coinPk}_i\big)$$

   Assuming that the above Midnight Commitment functions satisfy hiding when the nonce is picked at random, we can replace the recipient identity coinPk_i with the string 0 and no adversary can detect the change.
3. **Nullifers do not leak information about the recipients (PRF 5).** Nullifiers are computed as part of `receiveShielded` code, when the user sends a coin to the contract (in Withdraw and Swap flow). They are computed as follows:

$$\text{coinNull}(c, \text{sndr}) = \text{PersistantHash}\big(\text{cn-tag} \,\|\, \text{nonce} \,\|\, \text{color} \,\|\, \text{value} \,\|\, \text{sndr}\big)$$

   Assuming that the hash function PersistantHash behave as a PRF and assuming that nonce remains private throughout, the nullifer protects the user $\text{sndr}(\text{coinSk}_i)$.
4. **Additional ZSwap layer artifacts do not leak any information about the sender and receivers of a shielded transaction.** A ZSwap transaction contains several cryptographic layers that provides the correctness and functionality guarantees for a shielded payment to execute and land correctly. For instance, as part of the soundness of the payments, the value involved in the coins are also committed in Pedersen's commitments whose correctness is proved in zk via specifically designed Schnorr's zk proofs (Definition 18) (see [12]). Finally, the shielded transaction contains a ciphertext, that allows the recipient to detect that the coin generated in the transaction is for them. This ciphertext is computed with a *key-private* public key encryption scheme (See Definition 17. Key private means that the ciphertext does not leak any information about the public key that was used to compute it. This property is necessary for unlinkability.

Using the above properties, in Hybrid 1, we make the following changes in the way the experiment $\text{Exp-Unlink}_{\Pi,\mathcal{A}}$.

1. **Hybrid $\text{Hyb}_{1.1}$ Replace hones tproof with Simulated ZK proofs (use no witness)**: In $\text{Exp-Unlink}_{\text{Hyb}_{1.1},\mathcal{A}}$ we replace all the zk proofs created in the system (zk proofs of the circuits, and zk proofs of the SZSwap wallets), with *simulated* zk proofs $\text{ZK.Sim}_2(\text{crs}, \text{td}, x)$ (with $(\text{crs}, \text{td}) \leftarrow \text{ZK.Sim}_1(1^\lambda, R)$) that are calculated *without any secret.* as per Definition 8). This means that the transcript of every zk proof, *does not carry any information about the identity of the caller* of the circuits executed on Midnight.

If $\mathcal{A}$ wins the game Exp-Unlink$_{\text{Hyb}_{1.1},\mathcal{A}}$ with a probability $p_{1.1}$ that is non-negligible different than Exp-Unlink$_{\text{Hyb}_0,\mathcal{A}}$, then $\mathcal{A}$ can be used to break the zk property of each proof. Therefore, due the zero-knowledge Property (§ 8), it follows:

$$Pr[\text{Exp-Unlink}_{\text{Hyb}_{1.1},\mathcal{A}} \to 1] = p_{1.1} \approx_{\mu(\lambda)} p_0$$

2. **Hybrid** Hyb$_{1.2}$ **Remove Recipient Identity from Coin Commitments**: in this hybrid game, the challenger replaces all the commitments representing the coins that follow the form:

$$\text{coinCommit}(c, \text{rcpt}) = \text{HashK}\big(\text{cc-tag} \,\|\, \text{nonce} \,\|\, \text{color} \,\|\, \text{value} \,\|\, \text{rcpt}\big)$$

which takes as input the identity of the recipient rcpt, with the following:

$$\text{coinCommit}(\text{nonce} \,\|\, \text{color} \,\|\, \text{value} \,\|\, 0)$$

Note, in this experiment, the challenger will still maintains the private list of the raw coin-info (nonce, color, value) for each unspent coin. Such information, however, is not inputted in the commitment function.
If $\mathcal{A}$ wins the game Exp-Unlink$_{\text{Hyb}_{1.2},\mathcal{A}}$ with a probability $p_{1.2}$ that is non-negligible different than Exp-Unlink$_{\text{Hyb}_{1.1},\mathcal{A}}$, then $\mathcal{A}$ can be used to break the hiding of the commitment scheme used by Midnight shielded-coin layer. The reduction to hiding can go through since the proofs are simulated and since our protocol invokes the Midnight library with secret and random nonces for each new coin minted (evolved nonces are generated by Midnight). Therefore, assuming that coinCommit satisfies computationally hiding property (Definition 14, with CS.Gen(1^λ) outputing the parameters for the hash function) it follows that:

$$Pr[\text{Exp-Unlink}_{\text{Hyb}_{1.2},\mathcal{A}} \to 1] = p_{1.2} \approx p_{1.1}$$

3. **Hybrid** Hyb$_{1.3}$ Nullifier are computed without any sender's information (coinSk$_i$).
Namely, in Hyb$_{1.1}$ the challenge replaces this computation:

$$\text{coinNull}(c, \text{sndr}) = \text{PersistantHash}\big(\text{cn-tag} \,\|\, \text{nonce} \,\|\, \text{color} \,\|\, \text{value} \,\|\, \text{sndr}\big)$$

with this:

$$\text{coinNull}(c, \text{sndr}) = \text{PersistantHash}\big(\text{cn-tag} \,\|\, \text{nonce} \,\|\, \text{value} \,\|\, 0\big)$$

This computation is exactly as the coin commitment with the only difference in the tag cn-tag (vs cc-tag). Here nonce is the randomness used to compute the coin commitment is some earlier invocation of `mintShieldedToken`.
If $\mathcal{A}$ wins the game Exp-Unlink$_{\text{Hyb}_{1.3},\mathcal{A}}$ with a probability $p_{1.3}$ that is non-negligible different than Exp-Unlink$_{\text{Hyb}_{1.2},\mathcal{A}}$, then $\mathcal{A}$ can be used to break the security of Midnight's nullifers. This is part of the Midnight design and we defer the reader to their proofs of security.
Therefore,

$$Pr[\text{Exp-Unlink}_{\text{Hyb}_{1.3},\mathcal{A}} \to 1] = p_{1.3} \approx_{\mu(\lambda)} p_{1.2}$$

Hybrid Hyb$_{1.4}$

ZSwap Wallet Layer (Pedersen Commitments and Key-Private Public key encryption.)

A shielded payment involves the use of other cryptographic objects such as Pedersen's Commitments (Def. 13), Key-Private Public key Ciphertexts (Def. 17), and Schnorr's proofs (Def. 18), as described

in [12]. Pedersen's Commitments and Schnorr's proofs are used as part of the Midnight shielded-coin layer for the soundness of the proof, whereas Key-Private Public key Ciphertexts are used to guarantee discoverability of the coin to the receiver. In this hybrid experiments, we remove the secrets used in any of such objects. We can do so, since all zk proofs are simulated. Our code does not interfere with this layer, neither provides the randomness used in the wallet itself, so we inherit the security properties guaranteed by the Midnight shielded coin layer. The resulting transcript should be indistinguishable from the transcript generated in $\mathsf{Hyb}_{1.3}$, due to the zk property of Schnorr's zk proofs and the Key-Privacy of the Public key Ciphertexts.

Therefore,

$$Pr[\mathsf{Exp\text{-}Unlink}_{\mathsf{Hyb}_{1.4},\mathcal{A}} \to 1] = p_{1.3} \approx_{\mu(\lambda)} p_{1.3}$$

7.4.3 Hybrid 2. Replace $\mathsf{HashK}(\mathsf{userSk}_i, \cdot)$ with a Random Function RF_i (uses no secret)

In hybrid Hyb_2 we change the calculation of userComm and refundComm that are directly dependent on the user secret userSk_i. We replace the function $\mathsf{HashK}_{\mathsf{userSk}_i}$ keyed with the secret key of the user userSk_i, with a random function RF_i. Precisely, the following computations (See section 4.2):

$$\mathsf{userComm}(\mathsf{userSk})_i = \mathsf{HashK}(\text{“vault:user:”}, \mathsf{userSk}_i)$$
$$\mathsf{refundComm}(\mathsf{userSk}_i, \mathsf{reqId}) = \mathsf{HashK}(\text{“vault:refund:”}, \mathsf{userSk}_i, \mathsf{reqId}) \quad \text{the gate on refunds and swap settlement}$$

are replaced with:

$$\mathsf{userComm}^{\mathsf{RF}_i} = \mathsf{RF}_i(\text{“vault:user:”})$$
$$\mathsf{refundComm}^{\mathsf{RF}_i}(\mathsf{reqId}) = \mathsf{RF}_i(\text{“vault:refund:”}, \mathsf{reqId})$$

The only difference between the hybrid $\mathsf{Exp\text{-}Unlink}_{\mathsf{Hyb}_{1.4},\mathcal{A}}$ and $\mathsf{Exp\text{-}Unlink}_{\mathsf{Hyb}_{2},\mathcal{A}}$ is in the use of truly random functions $\mathsf{RF}_1(\cdot)$, $\mathsf{RF}_2(\cdot), \ldots, \mathsf{RF}_N(\cdot)$ instead of $\mathsf{HashK}_{\mathsf{userSk}_1}(\cdot)$, $\mathsf{HashK}_{\mathsf{userSk}_2}(\cdot)$, $\ldots, \mathsf{HashK}_{\mathsf{userSk}_N}(\cdot)$.

Assume that $\mathcal{A}$ wins the game $\mathsf{Exp\text{-}Unlink}_{\mathsf{Hyb}_{2},\mathcal{A}}$ (when HashK was used) with a probability p_2, that is non-negligible different from $\mathsf{Exp\text{-}Unlink}_{\mathsf{Hyb}_{1.4},\mathcal{A}}$ (where independent random functions RF_i are used), which is $p_{1.4}$. Then we can use $\mathcal{A}$ to build a distinguisher $\mathcal{B}$ that violates the multi-user PRF security of HashK (Definition 6) with the same probability.

Reduction $\mathcal{B}^{\mathcal{O}(\cdot,\cdot)}$

Activate adversary $\mathcal{A}$ and simulated game $\mathsf{Exp\text{-}Unlink}_{\mathsf{Hyb}_{1.4},\mathcal{A}}$ to it as follows:

- Simulate the execution of every oracle using the same approach as in Experiment $\mathsf{Hyb}_{1.4}$ with the following exception:
  1. For every new user U_i created for the cross-chain system, instead of picking the application secret key userSk_i, the challenger calls the oracle on input i: $\mathcal{O}(i, \cdot)$
  2. For every deposit flow for user U_i compute function userComm as:

$$\mathsf{userComm}^i = \mathcal{O}(i, \text{“vault:user:”})$$

3. For every withdraw, Swap, Supply, Redeem flow for user U_i compute function as

$$\text{refundComm}^i(\text{reqId}) = \mathcal{O}(i, \text{“vault:refund:”}||\text{reqId})$$

- Challenge

$$(\text{shAcct}_0,\ \text{shAcct}_1,\ \text{tokType},\ \text{amt},\ \text{aux}^*,\ \text{outcome}^*,\ \text{st}) \leftarrow \mathcal{A}^{\mathcal{O}}(\text{pp}),$$

where $\text{shAcct}_0, \text{shAcct}_1$ are honest shielded accounts.
Challenge

1. If shAcct_0 or shAcct_1 is not honest, output 0;
2. if $\text{vaultBal}[\text{shAcct}_0][\text{tokType}] < \text{amt}$ or $\text{vaultBal}[\text{shAcct}_1][\text{tokType}] < \text{amt}$, output 0;
3. otherwise, pick a random bit b and execute:

$$\text{Withdraw}(\text{shAcct}_b, \text{destAddr}^*, \text{tokType}, \text{amt}, \text{aux}\,;\ \text{privIn}_b)$$

for a fresh public-chain address destAddr^*, replacing the function HashK for shAcct_b with the evaluation of its correspondent oracle.

- Post-Challenge: simulate all the cross-chain oracles as above.
- Output: upon receiving a bit b' from $\mathcal{A}$.
- If $b' = b$ output 1, else output 0.

Analysis of $\mathcal{B}$'s probability of success:

- Case $\mathcal{O}(i, \cdot) = \text{RF}_i(\cdot)$.
  If the Oracle is implemented as a collection of truly random functions, then $\mathcal{B}$ is simulating the experiment $\text{Exp-Unlink}_{\text{Hyb}_2,\mathcal{A}}$. Hence,

$$Pr[\mathcal{B}^{\text{RF}_i(\cdot)} \to 1] = Pr[\text{Exp-Unlink}_{\text{Hyb}_2,\mathcal{A}} \to 1] = p_2$$

- Case $\mathcal{O}(i, \cdot) = \text{HashK}(sk_i, \cdot)$ If the Oracle is implemented as the function HashK evaluated on independently sampled secret keys userSk_i, then the transcript generated by $\mathcal{B}$ is identical to $\text{Exp-Unlink}_{\text{Hyb}_{1.4},\mathcal{A}}$. Hence,

$$Pr[\mathcal{B}^{\text{HashK}(sk_i,\cdot)} \to 1] = Pr[\text{Exp-Unlink}_{\text{Hyb}_{1.4},\mathcal{A}} \to 1] = p_{1.4}$$

Hence:

$$\left| Pr[\mathcal{B}^{\text{RF}_i(\cdot)} \to 1] - Pr[\mathcal{B}^{\text{HashK}(sk_i,\cdot)} \to 1] \right| = p_2 - p_{1.4}$$

Since we assume that HashK satisfies the property defined in Definition 6, it follows that $p_2 - p_{1.4}$ must be negligible.

We have removed the secret key of the caller from the authentication tags userComm and refundComm. In this hybrid world, the application secret key userSk_i is not used in *any computation*.

Note, however, that even if we are using a random function RF_i, the output is still deterministic – i.e.,

when queried with the same input, RF_i will return the same output. So, for instance, since in Deposit, $\mathsf{userComm}_i$ is computed on the same string "vaul:user' every time, the adversary can easily detect that the same user is performing various deposits. This is by design: we are not protecting the linkability of the deposit.

We are interested in guaranteed that withdrawals/swaps/supply/redeem are unlinkable to each other and unlinkable to the deposits. Recall that for these action, the user authenticate with $\mathsf{refundComm}_i$. And recall that:

$$\mathsf{refundComm}^{\mathsf{RF}_i}(\mathsf{reqId}) = \mathsf{RF}_i\big(\text{“vault:refund:”}, \mathsf{reqId}\big)$$

where reqId is unique per request (and $\mathsf{reqId} = \mathsf{HashP}(\mathsf{req})$).

This means that every $\mathsf{refundComm}_j$ is different from any other $\mathsf{refundComm}_j$ for any i, j, and carries no information about the recipient. Finally the request itself: req

1. req does not contain information about the identity of the withdrawer/swapper/supplier/redeemer.
2. each req is assumed to be unique due to the correctness of the underlying EVM nounce.

Putting everything together, in Hyb 2 we have:

(i) no identity is leaked from the cryptographic components. In the previous hybrid we removed the uses of the secret inputs from the coin and circuit computation, and in this hybrid we removed the identity userSk_i. So, in hybrid Hyb_2 we are left with no secret and no account identity is used in producing any component of the view: proofs are simulated, coin commitments carry a dummy recipient, nullifiers carry a dummy sender, and userComm and refundComm are evaluations of a random function;
(ii) no information identity is leaked from the withdraw/swap/supply/redeem request req (this is by design);
(ii) no information is leaked in the spending due to our consolidate-coin assumption (§ 7.3).

This means that in the world Hyb 2, the protocol messages are computed without any user-secret, the public transcript contain no information that identify the user who is chosen in the challenge phase by the challenger to execute Withdraw Consequently, in this world, no adversary can guess which user was chosen with a probability better than $\frac{1}{2}$. Hence:

$$\Pr[\mathsf{Exp\text{-}Unlink}_{\mathsf{Hyb}_2,\mathcal{A}} \to 1] = \frac{1}{2}$$

and due to the sequence of indistinguishable hybrid arguments, we conclude:

$$\Pr[\mathsf{Exp\text{-}Unlink}_{\mathsf{Hyb}_0,\mathcal{A}} \to 1] = \frac{1}{2} + \mu(\lambda)$$

□ □

7.5 Swap/Supply/ Redeem

The proof of Withdraw Unlinkability develops a sequence of arguments that removes *all user-dependent secrets* userSk_i *and* $\mathsf{shdAcct}_i$ *from every computation.* The final hybrid experiment of the

proof is a *simulated* world where no secrets are used (in any of the procedures). We notice that that Swap, Supply and Redeem follow the same structure of the Withdraw, and the same sequence of arguments apply. In the following we examine each call and explain why the same proof argument applies.

Swap. We notice that the circuits `startSwap`/ `completeSwap` follows the same structure of `startWithdraw`/ `completeWithdraw`. Specifically, `startSwap` is the same as `startWithdraw` with swapMap and swapView in place of reqMap and wdView. The logic is also almost identical: a coin is surrendered and burned in the same two moves, the same authentication tag refundComm$(\text{userSk}_b, \text{reqId})$ is computed. The only difference is that in swap, the transaction on the public chain involves only the vault account and concerns swapping coin *types*, whereas in the withdraw request, the transaction must have an explicit evm destination address destAddr. The settlement call `completeSwap` differs in two ways from `completeWithdraw`, however, neither of them depends on the user's identity. `completeWithdraw` mints on the *success* path rather than the failure path, and it mints *two* coins rather than one. Minting more coins does not change our arguments in $\text{Hyb}_{1.1}$, since in that hybrid we replace every proof in the transcript and $\text{Hyb}_{1.2}$ over *every coin commitment*, so a second commitment is replaced by the same hybrid and the same reduction. The amounts released and swapped, are in the public state, just like the amount withdrawn. Every step of Hyb_0 through Hyb_2 therefore applies verbatim, with Exp-LendUnlink in place of Exp-Unlink, and Definition 3.

Supply. Supply is a mirror of Swap with the following differences: (1) in Supply the token types and color are hardwired in the call (whereas in Swap they are an input chosen by the caller); since types and color are public in all the calls, this change does not affect the analysis. (2) Supply returns amtShares shares instead of an swapped amount. amtShares is not chosen by the challenger, it is determined by amt and by the market's exchange rate at the executing block, and the rate is a property of $\mathcal{L}_{\text{pub}}$, global to all suppliers, not of the shielded account that paid. Both differences do not affect the creation of the cryptographic objects and do not affect the analysis.

Redeem. Redeem is identical to Supply but with the color swapped, with the released amtAssets being a function of amtShares and the same global rate.

8 SOUNDNESS

In this section we analyze the soundness of the system. When defining soundness, we identify the soundness requirement for each action, and then defined the soundness of the system as the conjunction of the soundness of each properties. An alternative approach would have been to define an ideal functionality for our cross-chain system payment, and proving UC security of it, and is intended for future analysis.

This section is organized as follows: (1) Definition (Section 8.1): we define soundness of the system; (2) Assumptions (Section 8.2) we state the trust, model and cryptographic assumption we rely on to argue soundness (3) Proof (§ 8.4-§ 8.6) we prove that each action achieves soundness under the stated assumptions.

8.1 Definition of Soundness

For simplicity, in this first formal treatment, we define soundness of each procedure : Deposit, Withdraw, Swap, Supply, Redeem individually. The soundness of the overall system is then defined as the soundness of all of its procedures. In the future, we will consider using the Universal Composability framework to have a more comprehensive approach.

Definition 20. *Deposit Soundness:* $\Pi.\text{Deposit}^{\mathcal{L}_{\text{pub}},\mathcal{L}_{\text{shd}}}(\text{pubAddr}, \text{shdAcct}, \text{tokType}, \text{amt}, \text{aux}\,;\,\text{privIn})$ is *sound* if no PPT algorithm $\mathcal{A}$ can deposit amt, tokType on its own account shdAcct$*$ on $\mathcal{L}_{\text{shd}}$ and have transferred $\leq\ v$ of the same token to the vault on $\mathcal{L}_{\text{pub}}$ from its own account pubAddr with non-negligible probability.

Definition 21. *Withdraw Soundness:* $\Pi.\text{Withdraw}^{\mathcal{L}_{\text{pub}},\mathcal{L}_{\text{shd}}}(\cdot, \text{destAddr}*, \text{tokType}, \text{amt}, \text{aux}\,;\,\text{privIn})$ is *sound* if two conditions are satisfied: (1) eligibility: no PPT algorithm $\mathcal{A}$ can withdraw amt, tokType without controlling an account shdAcct* on Midnight that holds at least value (amt, tokType) with non-negligible probability. (2) no double-spending: no PPT algorithm $\mathcal{A}$, owning a shielded account shdAcct* with amount v^* can successfully withdraw amt of tokType from shdAcct*, and at its completion own an amount $\geq v^* -$ amt.

Definition 22. *Swap Soundness:* $\Pi.\text{Swap}^{\mathcal{L}_{\text{pub}},\mathcal{L}_{\text{shd}}}(\cdot, \text{tokType}, \text{amt}, \text{tokType}', \text{amt}', \text{aux}\,;\,\text{privIn})$ is *sound* if two conditions are satisfied: (1) eligibility: no PPT algorithm $\mathcal{A}$ can swap amt of tokType unless $\mathcal{A}$ controls an account shdAcct* that owns at least amt of tokType, with more than non-negligible probability. (2) no double-spending: no PPT algorithm $\mathcal{A}$, owning a shielded account shdAcct* with amount v_A^* of tokType$_A$ and v_B^* of tokType$'_B$ can successfully swap amt of tokType$_A$ for tokType$_B$ and then, upon swap completion own $\geq v_A^* -$ amt of tokType$_A$, with non-negligible probability.

Definition 23. *Supply soundness* $\Pi.\text{Supply}^{\mathcal{L}_{\text{pub}},\mathcal{L}_{\text{shd}}}(\text{shdAcct}^*, \text{tokType}, \text{amt}, \text{aux}; \text{privIn})$ is *sound* if two conditions are satisfied: (1) eligibility: no PPT algorithm $\mathcal{A}$ can supply amt of the underlying color color$_u$ (a function of tokType) without controlling an account shdAcct* on Midnight that owns at least amt of color$_u$, with more than non-negligible probability. (2) no double-spending: no PPT algorithm $\mathcal{A}$, owing a shielded account shdAcct* with amount v_U^* of color$_u$ can successfully supply amt of color$_u$ and, upon supply completion own $\geq v_U^* -$ amt of color$_u$ tokens, with non-negligible probability.

Definition 24. *Redeem soundness.* $\Pi.\text{Redeem}^{\mathcal{L}_{\text{pub}},\mathcal{L}_{\text{shd}}}(\text{shdAcct}, \text{tokType}', \text{amtShares}, \text{aux}\,;\,\text{privIn})$ is *sound* if no PPT $\mathcal{A}$ can redeem amtShares of coins of color$_w$ (a function of tokType$'$) without having supplied an amt of USD through Supply, such that amtShares = amt $+$ interest with non-negligible probability.

Definition 25 (**Cross-Chain Transaction System Soundness**)**.** A *public-to-private cross-chain Transaction system* Π, relative to $\mathcal{L}_{\text{pub}}$ and $\mathcal{L}_{\text{shd}}$ is *sound* it achieves Deposit, Withdraw, Swap, Supply and Redeem soundness.

8.2 Assumptions

In this proof we use the following assumptions

System Assumptions:

- Ethereum and Midnight are secure and sound instantiations of $\mathcal{L}_{\text{pub}}$ and $\mathcal{L}_{\text{shd}}$, respectively.
- MPC SigNet is trusted. It satisfies:
  1. **MPC committee integrity:** the committee signs only what the protocol authorises.

2. **Attestation faithfulness:** MPC only provide attestations for events that have been executed on $\mathcal{L}_{\text{pub}}$ (Ethereum). SigNet attests failed *only* for a request that did *not* execute on $\mathcal{L}_{\text{pub}}$.

Network Assumptions: Communication Channels are authenticated.

Cryptographic Assumptions:

- **ECDSA is EUF-CMA under additive key derivation and pre-signatures** (Definition 19). In our proof we use this property as a guarantee that no adversary forges the MPC attestation signature attSig that the settlement circuits verify against the pinned mpcRespKey. This assumption is also implictly used under the system assumption that Ethereum ($\mathcal{L}_{\text{pub}}$) is sound, so, no adversary forges a transaction signature under the vault's derived EVM account, which would move pooled funds with no Midnight call at all. Additive key derivation must be inside the game because the vault's account is a deterministic function of $(\text{vaultAddr}, \text{keyPath})$; pre-signatures must be inside it because the MPC produces signatures using a pre-processing step.
- HashP **is collision resistant** (Definition 7). We use this property for the computation of $\text{reqId} = \text{HashP}(\text{req})$, to claim that an attestation on reqId for req cannot be re-used to settle a different request req′ that collides with reqId. We also use in the computation, $\text{attDigest} = \text{HashP}(\text{reqId} \,\|\, \text{evmOut})$, to avoid that a collision lets a signature issued for one attested output be replayed as a signature for another.
- HashK **is a secure PRF** (Definition 5) **and therefore a MAC on fixed-length inputs** (Definition 16). Soundness uses the MAC direction: the caller gates accept only a witness userSk satisfying $\text{refundComm}(\text{userSk}, \text{reqId}) = \text{swapView}[\text{reqId}]$, so an adversary not holding userSk cannot claim another user's refund, and the same assumption protects the depositor gate $\text{userComm}(\text{userSk})$.
- **Midnight's shielded-circuit layer is sound** (Definition 10 and Definition 11). No adversary produces an accepting π for a transcript x that no honest execution of the circuit could have produced. This is what makes every assertion in the circuit boxes binding on the published state update, and what stops any authentication step to be skipped.
- **Midnight's shielded-coin and circuit-layer achieves soundness and zero-knowledge** (Definitions 10 and 15). Soundness of the Zswap proofs gives well-formedness and balance of the offer, so no transaction creates value except through a declared mint. Binding of coinCommit gives that a published commitment opens to at most one $(\text{nonce}, \text{color}, \text{value})$, so a surrendered coin cannot be re-opened at a larger value and a minted coin cannot be claimed at an amount other than the one the circuit fixed. Zero-knowledge allows us to claim that honest users' secret keys used as witness in computing the proof are protected.
- HashK **and** HashP **are collision resistant** (Definition 7). This property is vital in all authentication gates. For instance, an adversary who can find collisions in the digest of the requestID signed by SigNet, can easily re-use valid signatures, on requests that were never executed on $\mathcal{L}_{\text{pub}}$.
  Note that collision resistance implies one-wayness. We rely on the one-way property of HashK to prove that an authentication tag ($\gamma = \text{userComm}(\text{userSk})$ or $\gamma = \text{refundComm}(\text{userSk}, \text{reqId})$) published by an honest user cannot be used by an adversary to pass the authentication gate of the `complete` circuits.

8.3 Proof

Theorem 2. *If ECDSA is EUF-CMA under additive key derivation and pre-signatures signature scheme (Definition 19),* HashP *is a collision-resistant hash function (Definition 7),* HashK *is a secure*

PRF (Definition 5) and collision-resistant hash function, Midnight shielded-circuit layer achieves soundness (Definition 10) , Midnight shielded-coin layer (proofs and coin-commitments) satisfies soundness (Definition 10) and binding (Definition 15), then protocol Π described in Section 5 satisfy Soundness (Definition 25).

The proof of this theorem requires proving soundness of each action, from Deposit§ 20 to Supply§ 24.

The main observation is that every action Deposit, Withdraw, Swap, Supply, Redeem share the same logic and implementation, with minor differences that reflect only in the color of shielded transaction executed on $\mathcal{L}_{\text{shd}}$.

Every procedure follows the same template:

1. **Start Action:** Every action X is started with the `startX` circuit: `startDeposit`, `startWithdraw`, etc. Starting an action entails in the following steps
   - **Create EVM request** req and reqId: circuit crafts the transaction/call that must be executed on $\mathcal{L}_{\text{pub}}$. reqId $=$ HashP(req) is created.
   - **Differences with the latest release (erc20-vault-v0.4.0):** In the latest release, reqId is not created in the Start Action and there is no request counter reqCounter. The Start Action queues the request, the permissionless circuit `flushQueue` assigns its EVM nonce (the vault account's next nonce `vaultAccountNonce` for every action but Deposit, whose caller names the nonce of their own derived account), and the permissionless circuit `sendX` computes reqId $=$ HashP(req) over the completed request and notifies the MPC. Distinctness of reqId now follows from the EVM nonce, which the vault account never reuses, and a second identical request is refused while the first is open.
   - **Create authentication tag**: circuit creates userComm(userSk_i) (for `startDeposit`) or refundComm(userSk_i, reqId) for all other actions, is created.
     - * **Private Coins on $\mathcal{L}_{\text{shd}}$ are burned** (* Not present in `startDeposit`). In a Widthdraw/Swap/Supply/Redeem request, circuit privately sends their private coins to a burn address through the calls `receiveShielded` and `sendImmediateShielded`
   - reqId, refundComm **added to the map of pending requests for that action** At this point a pending action is identified by the pair (reqId, refundComm) (or (reqId, userComm) only for Deposit).
2. **Complete Action:** Every action X is completed with the `completeX` circuit. To complete an action identified by reqId, refundComm, the user must possess a valid attestation on reqId and outcome evmOut that verifies under SigNet's public key mpcRespKey. The outcome evmOut can be succeeded or failed. All complete circuits follows these steps:
   - **Verify Attestation (i.e., ECDSA signature).** The circuit runs the ECDSA verification algorithm for attestation attSig for input (reqId, evmOut) under public key mpcRespKey.
     For this check we use the unforgeability of ECDSA (in the pre-signature and additive-key derivation setting) and the collision-resistance properties of HashP.
   - **Check that** reqId **is pending.** The circuit check that reqId is pending in the action-dependent map (e.g., depositView, wdView, etc).
   - **Perform Action: if transaction was successful** evmOut= succeeded
     - ☐ **Verify authentication tag (**userComm/refundComm**).** The circuit recompute userComm(userSk_i) or refundComm(userSk_i, reqId) from the secret userSk_i and check if it is consistent with the one stored in the action-dependent map, keyed by reqId.
       For this check we rely on the one-wayness property of HashK, and the hiding properties of the Midnight layer.

- ☐ **Perform action.** For Deposit/Swap/Supply/Reedem the action is to **mint new coins**. For Withdraw no action is required.

■ **Refund is performed: if transaction failed** evmOut= failed.

- ☐ **Verify authentication tag (**userComm/refundComm**).** The circuit recompute userComm (userSk_i) or refundComm(userSk_i, reqId) from the secret userSk_i and check if it is consistent with the one stored in the action-dependent map, keyed by reqId. For this check we rely on the one-wayness property of HashK, and the hiding properties of the Midnight layer.
- ☐ **Refund.** For Withdraw/Swap/Supply/Reedem the action is to **mint new coins**. For Deposit no action is required.

Between Step 1 (start request) and Step 2 (complete request) the MPC system SigNet attempts to execute the request reqId on $\mathcal{L}_{\text{pub}}$. When the request is executed, SigNet signs the outcome (attestation).

Now, recall that at high-level, and $\mathcal{A}$ *violates soundness* if it is able to get out of the system more value than what they contribute in. In complete Deposit, this means to obtain shielded coins in $\mathcal{L}_{\text{shd}}$ for an amount v^* higher than what $\mathcal{A}$ actually deposited on $\mathcal{L}_{\text{pub}}$; in complete Withdraw this means to cash-out an amount v^* that is higher than what was burned in the request for withdrawing, etc. This also includes the case where $\mathcal{A}$ steals in-flight attestations that are intended for an honest user, and uses them to mint shielded transaction on its own account on $\mathcal{L}_{\text{shd}}$.

$\mathcal{A}$ can successfully execute `completeX` if and only if it presents (1) an attestation attSig, that verifiers under the SigNet's public key, on a pair $(\text{reqId}, \text{evmOut})$ (2) the request id reqId is in the map of pending requests, and (3) a proof of knowledge of the pre-image of the authentication tag (userComm (for Deposit) or refundComm for any other call associated to the request id reqId).

8.4 Soundness Arguments Common to all Circuits

Let $(\text{reqId}^*, \text{evmOut}^*, \text{attSig}^*)$ be what $\mathcal{A}$ presents to a `completeX` circuit, and suppose the circuit accepts. (Recall that in every `complete` action, the circuit will use reqId^* to look up the request req^* associated to it; where $\text{reqId} = \text{HashP}(\text{req})$, and attSig is a signature over $(\text{reqId}, \text{evmOut})$: i.e., $\text{attDigest}(\text{reqId}, \text{evmOut}) = \text{HashP}(\text{reqId} \parallel \text{evmOut})$.)

We now enumerate the ways that adversary $\mathcal{A}$ could violate soundness of the authentication patter of `complete`, and we rule them out, using the cryptographic assumptions that we rely on.

Case 1. $\mathcal{A}$ **claims a false outcome** evmOut^* **for her request** req^***.**

In this case, we consider an adversary that performed the `start` step, and created a request req^* with id reqId^* to be performed on $\mathcal{L}_{\text{pub}}$, and later it attempts to run a `completeX` circuit for a false outcome evmOut^*. False means, it does not reflect what truly happened on the chain, e.g., $\mathcal{A}$ could claim that the transactions failed when in reality it succeeded, or the opposite.

To pass the verification on a false evmOut^* that was not signed by the MPC (here we are using the *attestation faithfulness assumption* – SigNet only signs truth), $\mathcal{A}$ might have done one of the following:

1. **Forge** SigNet**'s attestation on** $\text{evmOut}^* \rightarrow$ **forge ECDSA signatures.** $\mathcal{A}$ could forge an attestation σ^* for the pair $\text{reqId}^*, \text{evmOut}^*$ of her choice, such that

$$\textbf{assert } \text{EcdsaVerify}\big(\text{attDigest}(\text{reqId}^*, \text{evmOut}^*),\ \sigma*,\ \text{mpcRespKey}\big) \rightarrow 1$$

regardless of whether the request transaction reqId^* was actually executed on Ethereum.
Assuming that ECDSA unforgeability with pre-signatures and key-derivation, this event happens with negligible probability.

2. **Re-use a valid signature on a previous input, by breaking the collision-resistance of** $\text{attDigest}(\cdot)$**.** Assume the MPC had signed an attestation for a pair $(\text{reqId}, \text{evmOut})$ at any point in time, and published attSig. If the adversary can craft a pair reqId^*, evmOut^* such that

$$\text{attDigest}(\text{reqId}, \text{evmOut}) = \text{attDigest}(\text{reqId}*, \text{evmOut}*)$$

the adversary, could *re-use* the valid signature attSig for $(\text{reqId}*, \text{evmOut}*)$.
Due to the collision-resistance property of HashP, this event happens with negligible probability.

3. **Re-use a valid signature on a previous input by break collision-resistance of** HashP**.** Similarly, an adversary could attempt to *re-use* attSig emitted on attDigest, by crafting a request req^*, such that $\text{HashP}(\text{req}^*) = \text{HashP}(\text{req})$ for an honest pending request req published on the chain. The adversary would intercept the signature attSig and present it to the `completeX` function before the honest user.
Due to the collision-resistance property of HashP, this event happens with negligible probability.

Case 3. $\mathcal{A}$ **uses a valid attestation for action A, to complete Action B.** $\mathcal{A}$ uses an attestation for an action (e.g., Swap) to settle a different action (e.g., Redeem). Specifically, $\mathcal{A}$ use the valid attestation of $(\text{reqId}, \text{evmOut})$ for a request $\text{reqId}^* = \text{HashP}(\text{req}^*)$ where req^* is a swap request, to pass the signature verification of `completeSupply`.

First note that in the implementation each requestID reqId lives in the data structure associated to the action requested. So, the reqId of swap request lives in the map swapView. When $\mathcal{A}$ attempts to run the Redeem circuit on $(\text{reqId}, \text{evmOut})$, the circuit check if reqId is pending in the map redeemView, which is not existing, and the circuit execution will fail – unless the adversary managed to create a swap request req_{swap} and a redeem request $\text{req}_{\text{redeem}}$ such that the hash $\text{HashP}(\text{req}_{\text{swap}}) = \text{req}_{\text{redeem}}$.

By the **collision-resistance** property of HashP, the probability of this attack is negligible.

Case 4. $\mathcal{A}$ **mints more than it paid.** amt and tokType are not chosen at settlement time: they are read from req, which was fixed at request time and whose digest is reqId. The same amt is the amount of the EVM transfer the committee signed. Changing it means finding a second record with the same reqId, which violated the **collision-resistance** property of $\text{HashP}()$.

Case 5. $\mathcal{A}$ **attempt to complete the same request twice (Replay Attack).** $\mathcal{A}$ replays a valid $(\text{reqId}, \text{evmOut}, \text{attSig})$. Every complete circuit removes both the record entry and the marker entry at reqId. A second call reads the same entries, and at apply time the node re-executes the transcript against real state, where they are gone. This replay attack is addressed at ledger level, and it does not rest on the protocol messages and cryptographic assumptions. **Differences with the latest release (erc20-vault-v0.4.0):** The latest release keeps this ledger-level protection (completion removes the request's queue entry, its attestation and its event) and adds an explicit mechanism that relies on timing information. Time is defined as the relative order of processed transactions on the public blockchain, and is tracked via the *block-height* of the *last-seen* transaction attested by the MPC: `flushQueue` stamps every request with the current `globalLastSeen` and raises `globalLastSeen` to the height of every attestation it folds, and an attestation settles a request only if its block height is strictly greater than the request's stamp.

Case 6. $\mathcal{A}$ claims an honest user's request. In this case, we consider the scenario where $\mathcal{A}$, observing the requests posted by honest users, and the attestations that the MPC posts back, attempts to use a valid attestation on behalf of the honest user. In other words, $\mathcal{A}$ tries to cash-in, swap, redeem, refund to her account, money that was deposited by an honest user.

An adversary could observe when a certain request reqId associated to a certain tag is executed on $\mathcal{L}_{\text{pub}}$; obtain the attestation on it from SigNet (recall that SigNet answers to anyone who asks), and use it to `completeX` the request on their *own* account on Midnight.

To successful execute a `completeX` (or `refundX`) circuit for requests posted by an honest user U_i, $\mathcal{A}$ should do one of the following:

1. **Pass the authentication check for markers** $\text{userComm}(\text{userSk}_i, \cdot)$ **(for** `completeDeposit`**) or** $\text{refundComm}(\text{userSk}_i, \cdot)$ **(for all other complete circuits)**
   To pass the authentication check present in any `complete` circuit, $\mathcal{A}$ must prove *knowledge of the pre-image* (userSk_i) of the authentication tag: $\text{userComm}(\text{userSk}_i, \cdot)$ or $\text{refundComm}(\text{userSk}_i, \cdot)$. We note that secret userSk_i is used in several messages of the protocol: not only to compute the authentication tags $\text{userComm}(\text{userSk}_i, \cdot)$ and $\text{refundComm}(\text{userSk}_i, \text{reqId})$, but also as witness in the zk proofs that honest user U_i has published over time.
   Due to the **zero-knowledge property** (Definition 12) of the Midnight shielded-circuit layer, $\mathcal{A}$ does not learn any information about userSk_i from the proofs generated by the user's honest circuit computation (with all but negligible probability).
   Due to the **pre-image resistance** (Definition 7) of HashK used to computed userComm and refundComm, the adversary cannot prove knowledge of the pre-image of those functions.
   Due to the **knowledge soundness property** (Definition 11), the $\mathcal{A}$ must know the witness (userSk_i) in order to generate a proof that verifies.

Next we analyze the attack vector that is specific to each call.

8.5 Soundness Argument Specific for Withdraw

In our protocol Withdraw (Procedure 5.3) consists of two steps: withdraw's request (Circuit `startWithdraw` C5) and withdraw's complete (Circuit `completeWithdraw` C7). In withdraw's request, a Midnight user holding account shdAcct requesting to withdraw $(\text{tokType}, \text{amt})$ to address $\text{destAddr}*$ is required to make a shielded payment to a *burn* address of the value $(\text{tokType}, \text{amt})$. This is the pre-requisite to claim a withdrawal. This payment is performed by calling Midnight circuits `receiveShielded` (surrCoin) and `sendImmediateShielded` $(\text{surrCoin}, \text{burnAddr}, \text{amtMax})$, where (surrCoin) contains the secret nounce associated to the coins that the user want to spend, and `sendImmediateShielded` requires knowledge of the secret key coinSk associated to the shdAcct that is attempting to withdraw. If both `receiveShielded` (surrCoin) and `sendImmediateShielded` are successful, then an authentication component refundComm is created for the request ID reqId associated to the withdraw request. refundComm will be used only if the withdrawal request is not completed successfully and this is indicated by an attestation attSig on a failed string. In such a case, the user executes and use attSig and the knowledge of the pre-image of refundComm as a proof of entitlement for a coin of value $(\text{tokType}, \text{amt})$.

A malicious user $\mathcal{A}$ violates withdraw soundness if they are able to either withdraw money that they do not own, or obtain a refund on withdrawal that was completed successfully. We analyze both cases, and show that they can only happen with negligible probability.

1. **Case 1.** $\mathcal{A}$ **attempts to withdraw money it does not own.** Recall, the withdraw request (req $\leftarrow$ (vaultAddr, reqCounter, keyVersion, keyPath, evmTx, chainId)) is considered by SigNet only if the user has successfully executed the circuit `startWithdraw`. This implies that circuits `receiveShielded` (surrCoin) and `sendImmediateShielded` (surrCoin, burnAddr, amtMax) have been successfully computed. Consider the following two cases:
   (a) $\mathcal{A}$ **owns a coin with amount** $\leq$ amtMax but still manages to successfully run `receiveShielded` (surrCoin) for the amount amt asserted in the circuit `startWithdraw`.
   Due to the **soundness** of the execution of the Midnight circuit `receiveShielded`, this case happens with negligible probability.
   (b) $\mathcal{A}$ **burns coins owned by an honest user.** An adversary $\mathcal{A}$ could observe honest user's executions of `receiveShielded` in multiple deposits/swaps/supply/redeem executions and attempt to use/manipulate the collected information into a successful execution of `receiveShielded` (surrCoin^H) with an honest coin surrCoin^H.
   Due to **the hiding, and zero-knowledge properties** of the primitives used by Midnight's library, this event could only happen with negligible probability.
2. **Case 2.** $\mathcal{A}$**'s withdraw request** reqId **is successfully executed on Ethereum** ***and*** **a refund on Midnight for** reqId**.** This case corresponds to **Case 1** analyzed in § 8.4.

8.6 Soundness Arguments Specific to Swap/Supply/Redeem Circuits

Swap/Supply/Redeem have the same logic. A user who owns amt coins of *type A* requests to swap them for amt′ coins of *type B*. They differ in three respects: the *admissible types*, how the returned amount is calculated, and how many coins the settlement mints; Swap mints two (the bought coin and the change), Supply and Redeem mint one. A fourth, smaller difference: `completeSwap` reads the stored request, while `completeSupply` and `completeRedeem` obtain the information from the attested outcome.

In Swap, user chooses the type *type B*, that she wants to swap to, and there is additional input amtMax, chosen by the user, which defines the most she is willing to spend She surrenders amtMax up front, the router spends some amtSpent $\leq$ amtMax, and `completeSwap` returns amtMax $-$ amtSpent as change. If the router would need more than amtMax, the EVM call reverts and the flow goes to `refundSwap`. In Supply, and Redeem, the types are fixed and hardwired in the contract definitions, and the amounts are chosen by the market. The types are defined in two addresses underlyingAddr and wrapperAddr written in the public state of the vault by `initialise` and are never chosen by the caller. In Supply, *Type A* is a fixed color_u and type B is the fixed color_w, and result of the call, the user gets amtShares of color_w. In Redeem, *Type A* is color_w and the amount redeemed is decided by the market and communicated to the circuit directly by the blockchain record. The Swap/Supply/Redeem follow the same authentication pattern of Widthdraw flow, for the refund path. As such, the arguments of soundness for Swap follow very closely the arguments on Withdrawal. The additional attack vector a malicious user can leverage in Swap/Supply/Redeem, is to initiate a request req for token types and amounts that do not correctly match the types and amounts of the coins it possesses.

We identify the following cases:

1. $\mathcal{A}$ **surrenders a coin that does not match the request it posted.** $\mathcal{A}$ could attempt to burn a coin of a cheap color, or of a value below amt, while filing a request for amt of an expensive tokType, and so withdraw on $\mathcal{L}_{\text{pub}}$ more than it gave up on $\mathcal{L}_{\text{shd}}$.
   This attack is prevented by the checks performed in the circuit. **assert** surrCoin.color = tokenTypeOf(domSep(tokType), vaultAddr), and surrCoin.value $=$ amtMax.

The surrendered coin's color and value are asserted *equal* to the values that go into req, and by binding of the coin commitment (Definition 15) the descriptor surrCoin the circuit checks is the same coin whose commitment and nullifier are published. By the **soundness of the zk argument system** (Definition 10) a proof that passes verification cannot correspond to an execution in which those assertions failed. Hence the event happens with probability negligible in λ.
The same case applies to Supply, with with $(\text{color}_u, \text{amt})$, and to Redeem with $(\text{color}_w, \text{amtShares})$. In all four the assertion is Step 2 of the request circuit and the argument is the same two-line appeal to binding and to circuit soundness.

2. $\mathcal{A}$ **surrenders a coin of the wrong color or value.** Prevented by Step 2 of Circuit C13 and Circuit C16. Note that the two colors here are *pinned*, not caller-supplied: color_u and color_w are functions of underlyingAddr and wrapperAddr read from state, so unlike `startWithdraw`, where the caller names tokType, there is no color argument to get wrong.
3. $\mathcal{A}$ **inflates the minted amount.** This is specific to Supply and Redeem only. `completeSupply` and `completeRedeem` take the minted value from evmOut and read no field of req. The only thing binding that value is that attSig verifies over attDigest$(\text{reqId}, \text{evmOut})$, so an inflated mint requires a forged or re-used attestation, excluded as above, *or* a committee that attests a share count the market did not return, which is excluded by assumption on the trustworthiness of the routing system and the chain, and not by the protocol.

9 TAKE AWAY FROM THIS DOCUMENT

9.1 Take away for a Midnight User

- **This cross-chain system involves a public chain so all amounts are public!**. Even if you can call actions such as Withdraw/Swap/Supply/Redeem anonymously on Midnight (i.e., without revealing your identity), the amounts of those actions must be executed on the public chain, so those amounts are still public, even if your identity is not.
- Because amounts are public, **you have to be careful when you choose the amounts and timing when performing Withdraw/Swap/Supply.** For instance, if you Deposit 5 USD, and then immediately Withdraw 5 USD on a fresh, and therefore unlinkable to the address you used to deposit, ethereum wallet, the amounts and timings still gives a way a link between the deposit and the withdraw. Similar arguments apply for Swap/Supply/Redeem.
- **Withdraw is unlikable to you, only if you provide a fresh Ethereum account.** If you withdraw funds into an account that you previously used, then the withdraw is connected to your previous activity. In the worst case, if you withdraw to an Ethereum account that you used to fund your derived address used in Deposit, then you have linked deposit and withdraw.
- **Keep your private coins consolidate in one coin.** The number of coins you need to collect in order to pay for a Withdraw/Swap/Supply leaks information about who you are. This is true regardless of whether you use our tool, or you simply operate on Midnight. You should always have your coins consolidated in one coin, and when you pay you should always have a change coin.
- **Deposit requests are all linkable to you.** Our system assigns a fixed identity $\text{userComm}(\text{userSk}_i)$ to each user that is used for each deposit. This makes all deposits requested by the owner of userSk_i linkable. (all other actions are not linkable since the authentication tag that we use, takes a one-time string reqId that makes each tag unique.)
- For privacy, **secret** userSk_i **must be kept secret at all times.** If someone learns userSk_i, they can

then link all your previous activities.

- On the bright side, if you leak userSk_i, the adversary can at most steal one pending request.
- Your anonymity set in Swap/Supply/Redeem is restricted to the number of people of are transacting on those particular coins.

9.2 For the Developers

- **A fixed** keyPath_i **per user is only a design choice.** In our implementation, the user's software picks a random application secret userSk_i as a permanent secret used across all deposits (and all calls). This design choice prioritizes user's convenience over stronger privacy. Convenience come from the fact that the user will only need to fund a single derived account (dictated by userComm_i). This gives weaker privacy because (1) all deposits to the user's account are strictly linkable – even if they came from different ethereum accounts. (2) leakage of the application secret userSk_i deanonymize all user's past activities.

9.3 For the Cryptographers

This treatment focuses on the erc20 vault contract, it is not an attempt to analyze the entire system rigorously. As such, we made a few simplifications, and some arguments are informal.

- Unlinkability: in our systems the identities of the caller are used in two layers: in our protocol messages and in Midnight's protocol messages. In this proof, we identify which Midnight's items contributed to our security and we rely on them, but we do not prove formally that those objects achieve the property that are claimed.
- Soundness: our definition of soundness could be made more comprehensive using the UC framework. We defer it to the next iteration of the document.

REFERENCES

[1] Elena Andreeva, Joan Daemen, Bart Mennink, and Gilles Van Assche. Security of keyed sponge constructions using a modular proof approach. In *Fast Software Encryption – FSE 2015*, volume 9054 of *Lecture Notes in Computer Science*. Springer, 2015.

[2] Mihir Bellare, Daniel J. Bernstein, and Stefano Tessaro. Hash-function based PRFs: AMAC and its multi-user security. In *EUROCRYPT 2016*, 2016. Full version: IACR ePrint 2016/142.

[3] Mihir Bellare, Alexandra Boldyreva, Anand Desai, and David Pointcheval. Key-privacy in public-key encryption. In *Advances in Cryptology – ASIACRYPT 2001*, volume 2248 of *Lecture Notes in Computer Science*, pages 566–582. Springer, 2001.

[4] Mihir Bellare, Alexandra Boldyreva, and Silvio Micali. Public-key encryption in a multi-user setting: Security proofs and improvements. In *Advances in Cryptology – EUROCRYPT 2000*, volume 1807 of *Lecture Notes in Computer Science*, pages 259–274. Springer, 2000.

[5] Eli Ben-Sasson, Alessandro Chiesa, Christina Garman, Matthew Green, Ian Miers, Eran Tromer, and Madars Virza. Zerocash: Decentralized anonymous payments from Bitcoin. In *IEEE Symposium on Security and Privacy*, 2014.

[6] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. On the indifferentiability of the sponge construction. In *Advances in Cryptology – EUROCRYPT 2008*, volume 4965 of *Lecture Notes in Computer Science*. Springer, 2008.

[7] Nir Bitansky, Ran Canetti, Alessandro Chiesa, and Eran Tromer. From extractable collision resistance to succinct non-interactive arguments of knowledge, and back again. In *ITCS 2012*, pages 326–349, 2012.

[8] Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *42nd Annual IEEE Symposium on Foundations of Computer Science (FOCS 2001)*, pages 136–145. IEEE Computer Society, 2001.

[9] Ran Canetti, Rosario Gennaro, Steven Goldfeder, Nikolaos Makriyannis, and Udi Peled. UC non-interactive, proactive, threshold ECDSA with identifiable aborts. In *ACM CCS 2020*, pages 1769–1787, 2020.

[10] Joan Daemen, Bart Mennink, and Gilles Van Assche. Full-state keyed duplex with built-in multi-user support. In *Advances in Cryptology – ASIACRYPT 2017*, volume 10625 of *Lecture Notes in Computer Science*. Springer, 2017. Full version: Cryptology ePrint Archive, Paper 2017/498, https://eprint.iacr.org/2017/498.

[11] Ivan Damgård. Commitment schemes and zero-knowledge protocols. In *Lectures on Data Security*, volume 1561 of *Lecture Notes in Computer Science*, pages 63–86. Springer, 1999.

[12] Felix Engelmann, Thomas Kerber, Markulf Kohlweiss, and Mikhail Volkhov. Zswap: zk-SNARK based non-interactive multi-asset swaps. *Proceedings on Privacy Enhancing Technologies*, 2022(4):507–527, 2022. ePrint 2022/1002, https://eprint.iacr.org/2022/1002.

[13] Oded Goldreich, Shafi Goldwasser, and Silvio Micali. How to construct random functions. *Journal of the ACM*, 33(4):792–807, 1986.

[14] Oded Goldreich and Yair Oren. Definitions and properties of zero-knowledge proof systems. *Journal of Cryptology*, 7(1):1–32, 1994.

[15] Lorenzo Grassi, Dmitry Khovratovich, Christian Rechberger, Arnab Roy, and Markus Schofnegger. Poseidon: A new hash function for zero-knowledge proof systems. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 519–535. USENIX Association, 2021. Full version: Cryptology ePrint Archive, Paper 2019/458, https://eprint.iacr.org/2019/458.

[16] Jens Groth. On the size of pairing-based non-interactive arguments. In *EUROCRYPT 2016*, pages 305–326, 2016.

[17] Jens Groth and Victor Shoup. On the security of ECDSA with additive key derivation and presignatures. In *EUROCRYPT 2022*, 2022.

[18] Jonathan Katz and Yehuda Lindell. *Introduction to Modern Cryptography*. CRC Press, 3rd edition, 2021.

[19] Thomas Kerber, Aggelos Kiayias, and Markulf Kohlweiss. KACHINA: Foundations of private smart contracts. In *2021 IEEE 34th Computer Security Foundations Symposium (CSF)*. IEEE, 2021.